

Python Exercise Book

Piotr Jastrzębski

2026-10-13

Table of contents

Python Exercise Book 4

I Introduction to Python 5

1 Variables, Types and Input/Output 6

2 Conditional Statements 9

3 Loops 11

4 Lists 14

5 Arrays 18

6 Tuples 21

7 Sets 25

8 Dictionaries 28

9 Functions 30

10 Function Arguments 33

11 f-strings 36

12 Exception Handling 38

13 File Handling 41
13.1 Text Files 41
13.2 Binary Files 43

14 Program Structure and Modules 44

15 Clean Code Principles 47

II Object-Oriented Programming 51

16 Simple Classes 52

17 Initializer 55

18 Properties 58

19 Inheritance and Polymorphism 61

20 Text Representations of Objects 64

21 Container Emulation 66

22 Arithmetic Emulation 68

23 Comparison Emulation 70

III Advanced Topics 72

24 Decorators 73

25 Regular Expressions 75

26 Iterators and Generators 77

27 Bitwise Operators 79

28 Selected Sequence Types 82

29 Lambda Expressions 84

30 The `functools` Module 86

Python Exercise Book

This is a collection of Python programming exercises. It was inspired by exercises created while teaching classes at the Faculty of Mathematics and Computer Science of the University of Warmia and Mazury in Olsztyn.

Some of the exercises were generated with ChatGPT, Claude, Gemini or DeepSeek.

Part I

Introduction to Python

1 Variables, Types and Input/Output

In the exercises in this chapter, use only sequences of statements — no conditional statements or loops.

1. Write a program that prints three lines: your first name, the name of your degree programme and the name of your university. Precede each line with a comment (`#`) describing what it prints.
2. Using a single call to the `print()` function, print the words `"Python"`, `"is"`, `"great"` so that they are separated by a hyphen (`-`) and the line ends with an exclamation mark instead of a newline character. Use the `sep` and `end` parameters.
3. Create the variables `first_name`, `age`, `height` and `is_student`, assigning to them, respectively: a string, an integer, a floating-point number and a Boolean value. Print each variable together with its type, using the `type()` function.
4. Read the user's first name using `input()` and print a greeting in the form `Hello, <name>!`.
5. Read two integers from the user (remember to convert them with `int()`), then print their sum, difference, product, quotient (`/`), integer quotient (`//`), remainder (`%`) and the result of exponentiation (`**`).
6. Evaluate and print the results of the following expressions:

```
7 / 2
7 // 2
-7 // 2
7 % 3
-7 % 3
2 ** 10
0.1 + 0.2
```

In comments, explain any results that surprised you.

7. Read the side length of a square (a floating-point number) from the user and print its area and perimeter. Round the results to two decimal places using the `round()` function.

8. Read a temperature in degrees Celsius from the user and convert it to degrees Fahrenheit using the formula $F = \frac{9}{5}C + 32$. Print the result.

9. Read a number of seconds (an integer) from the user and convert it to hours, minutes and seconds using the `//` and `%` operators. For example, for `3725` the program should print `1 h 2 min 5 s`.

10. Read a three-digit number from the user and print its hundreds, tens and units digits separately, using only the `//` and `%` operators. Then print the sum of these digits.

11. Create the variables `a = 5` and `b = 3`. Perform the following augmented assignment operations in turn: `a += b`, `a *= 2`, `a -= 1`, `a //= 3`, `a **= 2`, printing the value of `a` after each of them.

12. Check what happens when you try to add the string `"10"` to the number `5`. Read the error message and explain its cause in a comment. Then fix the code in two ways: so that the result is `15`, and so that the result is `"105"`.

13. Print the results of the following conversions: `int("42")`, `float("3.14")`, `int(3.99)`, `int(-3.99)`, `str(2026)`, `bool(0)`, `bool(15)`, `bool("")`, `bool("False")`. In comments, describe which values are treated as logically false.

14. You are given the string:

```
text = "  Introduction to Programming  "
```

Print: the length of the string (`len()`), the string without leading and trailing whitespace (`.strip()`), the string in uppercase (`.upper()`), the string in lowercase (`.lower()`) and the string in which the word `"Programming"` has been replaced with the word `"Python"` (`.replace()`).

15. You are given the string `word = "informatics"`. Print its first character, its last character (using a negative index) and the number of occurrences of the letter `"a"` (`.count()`). Also check what `word.find("ma")` returns.

16. Read the user's first name and last name (two separate calls to `input()`) and print their initials in the form `J.K.`, using string indexing and the `+` operator.

17. Using the `*` operator for strings, print a line made up of 40 `=` characters, below it the title `REPORT`, and below that another line of 40 `=` characters.

18. Read an amount in PLN (a floating-point number) and the euro exchange rate from the user. Print the amount converted to euros, rounded to two decimal places.

19. Read the coordinates of two points in the plane from the user: `x1`, `y1`, `x2`, `y2`. Calculate and print the distance between the points, using the `** 0.5` operator instead of a square root function.

20. Write a program that calculates the BMI (body mass index). Read the body weight in kilograms and the height in centimetres, convert the height to metres, calculate $BMI = \frac{weight}{height^2}$ and print the result rounded to one decimal place. Make sure the prompts shown when reading the data are clear.

2 Conditional Statements

1. Read an integer from the user and print whether it is positive, negative or equal to zero.
2. Read an integer from the user and print whether it is even or odd (use the `%` operator).
3. Read the user's age and print the message `"Adult"` or `"Minor"`. Then write the same solution in a single line using a conditional expression (`x if condition else y`).
4. Read two numbers from the user and print the larger one without using the `max()` function. If the numbers are equal, print an appropriate message.
5. Read three numbers from the user and print the largest of them, using only `if`, `elif` and `else` statements and comparison operators.
6. Read from the user the number of points (from 0 to 100) scored in a test and print the grade according to the scale:

   - 0–50 — `2.0`,
   - 51–60 — `3.0`,
   - 61–70 — `3.5`,
   - 71–80 — `4.0`,
   - 81–90 — `4.5`,
   - 91–100 — `5.0`.

   If the number of points is out of range, print a message about invalid input.
7. Read a year from the user and check whether it is a leap year. A year is a leap year if it is divisible by 4 and not divisible by 100, or if it is divisible by 400. Write the condition as a single expression using the `and` and `or` operators.
8. Read a temperature from the user and, using a chained comparison (e.g. `15 <= t < 25`), print whether it is `"cold"` (below 15), `"pleasant"` (from 15 to 25) or `"hot"` (above 25).
9. Read a single character from the user and check whether it is a vowel (`a`, `e`, `i`, `o`, `u`), using the `in` operator with the string `"aeiou"`. Handle uppercase letters as well (e.g. with `.lower()`).
10. Read the lengths of three line segments from the user and check whether they can form a triangle (the sum of the lengths of any two sides must be greater than the third). If so, determine whether the triangle is equilateral, isosceles or scalene.

11. Read the coefficients `a`, `b`, `c` of the quadratic equation $ax^2 + bx + c = 0$ from the user. Calculate the discriminant Δ and print the roots of the equation (two, one, or a message that there are no real roots). Handle the case `a == 0` separately.

12. Read the coordinates `x`, `y` of a point from the user and print which quadrant of the coordinate system the point lies in. Handle the cases where the point lies on an axis or at the origin.

13. Read a login and a password from the user. If the login is `"admin"` and the password is `"python123"`, print `"Logged in"`. Otherwise, print whether the login or the password was incorrect. Use a nested `if` statement.

14. Read a string from the user and print whether it is empty. Use the fact that an empty string has the Boolean value `False` (the condition `if not text:`).

15. Read the number of a day of the week (1–7) from the user and print its name, using a `match`–`case` statement. For values out of range, use the `case _` pattern.

16. Write a simple calculator: read two numbers and an operation symbol (`+`, `-`, `*`, `/`) and print the result. Use a `match`–`case` statement to select the operation. Handle division by zero and an unknown operation symbol with a message to the user.

17. Read the total purchase amount from the user. The shop offers a discount: 5% for amounts above 100 PLN, 10% for amounts above 500 PLN and 15% for amounts above 1000 PLN. In addition, if the customer has a loyalty card (read the answer `y`/`n`), they receive an extra 2% discount. Print the amount to be paid.

18. Read from the user the weight of a parcel in kilograms and whether it is a priority parcel. Calculate the shipping cost according to the following rules:

    - up to 1 kg — 10 PLN,
    - above 1 kg up to 5 kg — 15 PLN,
    - above 5 kg up to 20 kg — 25 PLN,
    - parcels above 20 kg are not accepted.

    A priority parcel costs 50% more.

19. Read a date from the user as three numbers: day, month, year. Check whether the date is valid (take into account the number of days in each month and leap years).

20. Read a time in 24-hour format from the user (`hour` and `minute` as two integers) and print it in 12-hour format with the suffix `AM` or `PM` (e.g. `0:15` → `12:15 AM`, `13:05` → `1:05 PM`, `12:00` → `12:00 PM`). Validate the input data.

3 Loops

In the exercises in this chapter, do not use lists — variables, strings and the `range()` function are enough.

1. Using a `for` loop and the `range()` function, print the numbers from 1 to 10, each on a separate line. Then print them on a single line, separated by spaces (the `end` parameter of the `print()` function).
2. Using a `for` loop, print:

   - all even numbers from 0 to 30,
   - the numbers from 20 to 0 in descending order,
   - every fifth number in the range from 5 to 100.

   In each case, use appropriate arguments of the `range()` function instead of a conditional statement.
3. Read a natural number `n` from the user and calculate the sum of the numbers from 1 to `n` using a loop. Compare the result with the formula $\frac{n(n+1)}{2}$.
4. Read a natural number `n` from the user and calculate its factorial $n!$ using a `for` loop.
5. Read an integer from the user and print its multiplication table from 1 to 10 in the form `7 x 3 = 21`.
6. Using nested loops, print a 10×10 multiplication table in the form of a table. To align the columns, you can use the string method `.rjust()` or the tab character `"\t"`.
7. Read a string from the user and, using a `for` loop that iterates over the characters of the string, count how many vowels and how many digits it contains.
8. Read a string from the user and build (in a loop, character by character) a new string in reverse order. Do not use slicing.
9. Read a natural number from the user and, using a `while` loop and the `//` and `%` operators, calculate the sum of its digits and the number of digits.
10. Write a program that reads numbers from the user until `0` is entered. At the end, print the sum, the number of values read (excluding the zero) and their arithmetic mean. Handle the case where the first number entered is `0`.

11. Keep reading a number in the range from 1 to 10 from the user until they enter a valid value (input validation in a `while` loop). Print a message after each invalid attempt.

12. Print all the numbers from 1 to 100, but instead of numbers divisible by 3 print `Fizz`, instead of numbers divisible by 5 print `Buzz`, and instead of numbers divisible by both 3 and 5 print `FizzBuzz`.

13. Read a natural number `n` from the user and check whether it is a prime number. Exit the loop with a `break` statement once the first divisor has been found. Use the `else` clause of the `for` loop to print a message that the number is prime.

14. Print all prime numbers less than 100, using nested loops.

15. Using a `for` loop and the `continue` statement, print the numbers from 1 to 50, skipping numbers divisible by 7 and numbers containing the digit 7 (check `"7" in str(i)`).

16. Read a natural number `n` from the user and print the first `n` terms of the Fibonacci sequence, using only two helper variables and multiple assignment (`a, b = b, a + b`).

17. Read two natural numbers from the user and calculate their greatest common divisor using the Euclidean algorithm (the version with the remainder of division), using a `while` loop. Based on this, also calculate their least common multiple.

18. Read a natural number `n` from the user and print (using loops and the `*` operator for strings) the following shapes of height `n`:

    - a right triangle made of asterisks, aligned to the left,
    - the same triangle aligned to the right,
    - a Christmas tree (an isosceles triangle).

    Example of a Christmas tree for `n = 3`:

    ```
      *
     ***
    *****
    ```

19. Read a natural number from the user and print the successive terms of the Collatz sequence until the sequence reaches the value 1 (if the number is even, divide it by 2; otherwise multiply it by 3 and add 1). At the end, print the number of steps performed and the largest value that appeared in the sequence.

20. Write a number guessing game. Set a secret number in the range 1–100 in the code (e.g. `secret = 37`). The user has at most 7 attempts; after each attempt the program prints `"too low"` or `"too high"`. Once the number has been guessed, print the number of attempts used and exit the loop with a `break` statement. If all attempts are used up, print the secret number (use the `else` clause of the loop).

21. Write a program with a text menu running in a `while True` loop. The program stores an account balance (initially 0 PLN) and lets the user choose an option: `1` — deposit, `2` — withdrawal, `3` — display balance, `0` — exit. Do not allow a withdrawal greater than the balance. Exit the loop with a `break` statement.

4 Lists

1. Create a list `fruits` containing five fruit names (as strings). Print the whole list, then print the second and the last element separately.
2. Create an empty list `shopping_list`. Add four products to it using the `.append()` method, then insert a fifth product at the beginning of the list using `.insert()`. Print the result.
3. You are given the list:

```
colors = ["red", "green", "blue", "yellow", "green", "purple"]
```

Remove the first occurrence of `"green"` using `.remove()`, then remove the element at index `3` using `del`. Print the list after each operation.

4. Create a list `scores` containing any 8 integers. Print the number of elements (`len()`), their sum (`sum()`) and their arithmetic mean (calculated manually from `sum` and `len`).
5. You are given the list:

```
letters = ["a", "b", "c", "d", "e", "f", "g", "h"]
```

Using only slicing: - get the first three elements, - get every second element, - get the elements from index `2` to `5` (inclusive), - reverse the list.

6. You are given the list:

```
temperatures = [23.1, 18.4, 30.0, 15.7, 22.2, 28.9]
```

Print the list sorted in ascending order (without modifying the original — use `sorted()`), then sort the original in descending order in place (`.sort()`). Print both versions.

7. Create a list `squares` containing the squares of the numbers from 1 to 15 inclusive, using a list comprehension. Then create a second list `even_squares` that contains only those squares that are even numbers.
8. You are given the list:

```
words = ["apple", "hi", "banana", "go", "strawberry", "no", "watermelon"]
```

Using a list comprehension, create a new list `long_words` that contains only the words longer than 4 characters.

9. Create a two-dimensional 3×3 list `matrix` filled with the numbers from 1 to 9. Print:

- the whole row at index `1`,
- the element in row `2`, column `0`,
- all elements on the main diagonal (indices `[0][0]`, `[1][1]`, `[2][2]`).

10. You are given two lists:

```
student_names = ["Alice", "Bob", "Charlie", "Diana"]
student_grades = [85, 92, 78, 95]
```

Combine them into a list of tuples `student_records` using `zip()`. Then print the names of the students who received a grade higher than 80.

11. You are given the list:

```
cities = ["Warsaw", "Berlin", "Prague", "Vienna", "Budapest", "Prague", "Rome"]
```

Using `enumerate()`, print all the indices at which the value `"Prague"` occurs.

12. Create a nested list:

```
original = [[1, 2], [3, 4], [5, 6]]
```

Make a shallow copy (`shallow_copy`) and a deep copy (`deep_copy`). Modify the element `original[0][0] = 99`. Print all three lists and explain (with a comment in the code) why the results differ.

13. You are given the list:

```
nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9], [10]]
```

Write two solutions that flatten it into a single list `flat_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`:

- using a nested list comprehension,
- using a `for` loop.

14. You are given a list of numbers:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

Write a function `chunk_list(data, chunk_size)` that splits the list into smaller sublists of the given size. Example call:

```
chunk_list(numbers, 4)
# expected result: [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

15. You are given a matrix (a list of lists):

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Write code that creates `transposed_matrix` — the transpose of the matrix above — using `zip()` and a list comprehension. Print the result.

16. You are given the list:

```
raw_data = [3, 5, 2, 3, 7, 5, 8, 2, 9, 7, 1, 3]
```

Create a list `unique_data` that contains the same elements without duplicates, preserving the original order of first occurrence. Do not use `set()` in a way that changes the order.

17. You are given a list of strings:

```
raw_prices = ["12.50", "abc", "7.99", "", "25.00", "not_a_number", "3.14"]
```

- Using a helper function and `filter()`, filter out the elements that cannot be converted to `float`.
- Using `map()`, convert the filtered elements to the `float` type.
- Store the result in the list `valid_prices`.

18. Using only Python's built-in list and its methods (`.append()`, `.pop()`, `.pop(0)`):

    - Simulate a stack (LIFO) — push 5 elements, then pop 3.
    - Simulate a queue (FIFO) — enqueue 5 elements, then dequeue 3.

    Print the state of the structure after each operation.

19. You are given a list of dictionaries:

```
employees = [
    {"name": "Alice", "department": "IT", "salary": 7200},
    {"name": "Bob", "department": "HR", "salary": 5400},
    {"name": "Charlie", "department": "IT", "salary": 6800},
    {"name": "Diana", "department": "HR", "salary": 5900},
    {"name": "Eve", "department": "IT", "salary": 7200},
]
```

Sort the list:

- in ascending order by `department`, and within the same department — in descending order by `salary`,
- use the `key` parameter of the `sorted()` function with a `lambda` expression.

20. Write two variants of code that find the first number divisible by both 7 and 13 in the range from 1 to 10,000,000:

- List comprehension — first generates the full list, then takes the first element.
- Generator expression — uses `next()` on a generator expression.

Measure the execution time of both variants using the `time` module and print the results. In a comment, explain why their performance differs.

5 Arrays

In the exercises in this chapter, use the `array` type from the standard `array` module (`from array import array`). Unlike a list, an array stores elements of only one fixed numeric type, specified by a type code (e.g. `'i'` — integers, `'d'` — floating-point numbers).

1. Create an array `numbers` with the type code `'i'` containing the numbers `[4, 8, 15, 16, 23, 42]`. Print the array, its length, and its first and last elements.
2. Create an array `measurements` with the type code `'d'` containing any five measurements. Print its `typecode` and `itemsize` attributes (the size of a single element in bytes). Compare `itemsize` with that of an array with the type code `'i'`.
3. Create an empty array with the type code `'i'`. Using a `for` loop and the `.append()` method, add the squares of the numbers from 1 to 10 to it. Then, using the `.extend()` method, add the numbers from the list `[200, 300, 400]` all at once. Print the result.
4. You are given the array:

```
values = array('i', [10, 20, 30, 40, 50, 60, 70])
```

Using slicing, print: the first three elements, every second element, and the elements in reverse order. Check the type of the result of slicing.

5. For the array from the previous exercise:

   - change the element at index `2` to `35`,
   - insert the number `5` at the beginning of the array (`.insert()`),
   - remove the first occurrence of the number `40` (`.remove()`),
   - remove and print the last element (`.pop()`).

   Print the array after each operation.

6. Create an array with the type code `'i'` and try to add to it:

   - the floating-point number `3.5`,
   - the string `"7"`.

Read the error messages and explain in comments how an array differs from a list in this respect. Also check what happens when you add an integer to an array with the type code `'d'`.

7. Create an array with the type code `'b'` (signed 1-byte integers) and try to add the numbers `127`, `128` and `-129` to it. Describe in a comment where the limit on the range of values comes from.

8. You are given the array:

```
grades = array('i', [3, 5, 4, 5, 2, 5, 4, 3, 5])
```

Print how many times the grade `5` occurs (`.count()`), the index of the first occurrence of the grade `2` (`.index()`), the sum and the mean of the grades, and the highest and lowest grade (`max()`, `min()`).

9. You are given the list `prices = [19.99, 5.49, 3.20, 12.00, 7.75]`. Create an array with the type code `'d'` from it, then in a loop increase each price by 23% (VAT), modifying the array elements by index. Finally, convert the array back to a list using the `.tolist()` method and print the result.

10. You are given the array:

```
temperatures = array('d', [12.5, 14.1, 9.8, 15.3, 17.0, 11.2, 13.4])
```

Using a `for` loop, find (without using `max()` and `min()`) the index of the day with the highest and the lowest temperature. Also count on how many days the temperature was higher than the mean.

11. Create an array with the type code `'i'` containing the numbers from 1 to 20. Using a list comprehension, create from it a new array containing only the numbers divisible by 3. Reverse it in place using the `.reverse()` method and print the result.

12. You are given two arrays of the same length:

```
a = array('d', [1.0, 2.0, 3.0])
b = array('d', [4.0, 5.0, 6.0])
```

Calculate and print:

- an array `c` that is the sum of the corresponding elements,
- the dot product of the vectors `a` and `b`.

Use `zip()`. Also check what the expression `a + b` gives and why it is not a vector sum.

13. Keep reading integers from the user until they enter an empty string, and store them in an array with the type code `'i'`. Then sort the elements (the `sorted()` function returns a list — create a new array from it) and print the median.

14. Create an array `data_array` (type code `'i'`) containing the numbers from 0 to 9999 and print how many bytes the data stored in it occupies (`len(data_array) * data_array.itemsize`). In a comment, explain why a list with the same numbers takes up much more memory (hint: what does a list actually store?).

6 Tuples

1. Create a tuple `weekdays` containing the names of the five working days (as strings). Print the whole tuple, then print the first and the last element separately.
2. Create an empty tuple `empty_tuple` and a single-element tuple `single_item` storing the number `42`. Print both tuples together with their types (`type()`). Check what happens if you omit the comma in the single-element tuple, and explain the result in a comment.
3. You are given the tuple:

```
months = ("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
```

Using only slicing: - get the summer months (Jun–Aug), - get every third month, - get the last four months, - reverse the order of the elements.

4. You are given the tuple:

```
coordinates = (52.23, 21.01)
```

Try to change the second element to `19.94`. Catch the `TypeError` exception using a `try/except` block and print a message explaining why the operation failed.

5. You are given the tuple:

```
grades = (4, 5, 3, 5, 4, 5, 2, 4, 5, 3)
```

Print how many times the grade `5` occurs (`.count()`), and give the index of the first occurrence of the grade `2` (`.index()`).

6. You are given the tuple:

```
person = ("Alice", 30, "developer", "Warsaw")
```

Unpack it into the variables `name`, `age`, `role`, `city` in a single line. Then unpack it again, but this time assign only the first name to `name` and the rest to the variable `rest` using the `*` operator.

7. Create two variables `x = 10` and `y = 20`. Swap their values in a single line, using tuple packing/unpacking (without a temporary variable). Print the values before and after the swap.

8. Create a dictionary `city_distances` in which the keys are tuples of two cities and the values are distances in kilometres, e.g.:

```
("Warsaw", "Berlin"): 573
```

Add at least 4 entries. Write code that, for a given pair of cities, prints the distance or a message that the route does not exist.

9. You are given the tuple:

```
products = ("laptop", "mouse", "keyboard", "monitor", "headset")
```

Using `enumerate()` with the parameter `start=1`, print a numbered list of products in the format:

```
1. laptop
2. mouse
...
```

10. You are given two lists:

```
country_names = ["Poland", "Germany", "France", "Spain"]
country_codes = ["PL", "DE", "FR", "ES"]
```

Combine them into a tuple of tuples `country_data` using `zip()` wrapped in `tuple()`. Print the result and, separately, the third element.

11. You are given a list of tuples:

```
runners = [("Alice", 12.5), ("Bob", 11.8), ("Charlie", 13.1), ("Diana", 11.2)]
```

Sort the list in ascending order by time (the second element of the tuple), using `sorted()` and the `key` parameter with a `lambda` expression. Print the sorted list.

12. Import `namedtuple` from the `collections` module. Create a type `Book` with the fields `title`, `author`, `year`, `pages`. Create three book instances and put them in the list `library`. Print the title and author of each book by iterating over the list.

13. Write a function `analyze_numbers(data)` that takes a tuple of numbers and returns a tuple of three values: the minimum, the maximum and the arithmetic mean. Call the function and unpack the result into three variables `min_val`, `max_val`, `avg_val`.

14. You are given the tuple:

```
original = (10, 20, 30, 40, 50)
```

Convert the tuple to a list, add the element `60`, remove the element `20`, sort in descending order, and then convert it back to a tuple `modified`. Print the input tuple and the resulting tuple.

15. You are given a structure storing data about students:

```
classroom = (
    ("Alice", (90, 85, 92)),
    ("Bob", (78, 82, 88)),
    ("Charlie", (95, 91, 87))
)
```

Write code that prints the name of each student together with the mean of their grades, using only indexing and a `for` loop.

16. You are given the tuples:

```
version_a = (2, 8, 1)
version_b = (2, 8, 3)
version_c = (2, 10, 0)
version_d = (3, 0, 0)
```

Print the results of the comparisons `version_a < version_b`, `version_b < version_c` and `version_c < version_d`. In a comment, explain the principle of lexicographic comparison of tuples.

17. Create a list of tuples representing the coordinates of points:

```
raw_points = [(1, 2), (3, 4), (1, 2), (5, 6), (3, 4), (7, 8)]
```

Convert the list to a `set` to remove duplicates, and then convert the result back to a sorted list of tuples `unique_points`. Print the result and explain in a comment why tuples can be elements of a set, while lists cannot.

18. You are given a list of dictionaries:

```
sensors = [
    {"id": "S1", "temp": 22.5, "humidity": 45},
    {"id": "S2", "temp": 19.8, "humidity": 60},
    {"id": "S3", "temp": 25.1, "humidity": 38},
    {"id": "S4", "temp": 18.3, "humidity": 72},
]
```

Using a generator expression and `tuple()`, create a tuple `high_humidity` containing tuples `(id, humidity)` only for the sensors whose humidity exceeds 50.

19. Import the `sys` module. Create a list `data_list` and a tuple `data_tuple` containing the same 1000 integers (from 0 to 999). Compare the size of both structures using `sys.getsizeof()`. Print the results and explain in a comment why the tuple takes up less memory.

20. Write three functions that operate only on tuples (without converting them to a list):

    - `tuple_replace(data, index, value)` — returns a new tuple in which the element at the given index has been replaced with `value` (use slicing and tuple concatenation).
    - `tuple_insert(data, index, value)` — returns a new tuple with the element inserted at the given position.
    - `tuple_remove(data, index)` — returns a new tuple without the element at the given index.

    Test each function on the tuple `sample = (10, 20, 30, 40, 50)` and print the results.

7 Sets

1. Create a set `colors` containing five colour names. Print the set and its size (`len()`). Run the program several times and check whether the order of the printed elements is always the same as in the definition.
2. Create an empty set `visited`. Check the type of the expression `{}` and explain in a comment why an empty set must be created with `set()`. Then add three city names to the set using the `.add()` method, including one city twice. Print the result.
3. You are given the list:

```
numbers = [3, 7, 3, 1, 9, 7, 7, 2, 1, 3]
```

Using a set, print the number of distinct values in the list and a sorted list of these values (`sorted()`).

4. You are given the set `fruits = {"apple", "banana", "cherry"}`. Remove the element `"banana"` from it using the `.remove()` method, then try to remove the element `"mango"`, first with the `.remove()` method and then with `.discard()`. Describe the difference between these methods in a comment.
5. Read a word from the user and use the `in` operator to check whether it belongs to the set of reserved words:

```
keywords = {"if", "else", "elif", "for", "while", "def", "return", "import", "class"}
```

6. You are given the sets:

```
python_students = {"Alice", "Bob", "Charlie", "Diana", "Eve"}
java_students = {"Charlie", "Eve", "Frank", "Grace"}
```

Print (using the `|`, `&`, `-`, `^` operators):

- the students learning at least one language,
- the students learning both languages,
- the students learning only Python,
- the students learning exactly one language.

Then write the same operations using the `.union()`, `.intersection()`, `.difference()` and `.symmetric_difference()` methods.

7. Read two strings from the user and print:

   - the letters that occur in both strings,
   - the letters that occur only in the first string,
   - the number of distinct letters used in both strings combined.

8. Read a sentence from the user and check whether it is a pangram, i.e. whether it contains all the letters of the English alphabet (`"abcdefghijklmnopqrstuvwxyz"`). Letter case does not matter. If the sentence is not a pangram, print the missing letters in alphabetical order.

9. You are given the sets:

```
required_skills = {"python", "sql", "git"}
candidate_a = {"python", "sql", "git", "docker"}
candidate_b = {"python", "excel"}
```

Using the `.issubset()` and `.issuperset()` methods (or the `<=`, `>=` operators), check which candidate has all the required skills. For the candidate who does not, print the missing skills. Also use the `.isdisjoint()` method to check whether candidate B has any of the required skills at all.

10. Using a set comprehension, create the set of remainders of dividing the squares of the numbers from 1 to 50 by 7. Print the sorted result.

11. You are given a list of words:

```
words = ["Python", "java", "python", "C", "JAVA", "Rust", "c", "rust", "Go"]
```

Create a set `unique_languages` containing the language names without repetitions, ignoring letter case (written in lowercase). Print the number of distinct languages.

12. You are given the list:

```
raw_data = [5, 3, 5, 1, 3, 8, 1, 9, 8]
```

Create a list `unique_data` without duplicates, preserving the order of first occurrence. To check whether an element has already occurred, use a helper set `seen`.

13. You are given the list:

```
numbers = [4, 9, 1, 4, 7, 9, 2, 4, 1]
```

Using two sets (`seen` and `duplicates`) and a single loop, determine the set of values that occur in the list more than once.

14. You are given two lists of order IDs:

```
orders_january = [101, 102, 103, 104, 105, 106]
orders_february = [104, 105, 106, 107, 108]
```

Print the orders placed only in January, only in February, and in both months. Print the result of each operation as a sorted list.

15. Create a set of tuples representing the coordinates of the squares visited by a robot:

```
moves = ["R", "R", "U", "L", "D", "D", "R", "U", "U"]
```

The robot starts at square `(0, 0)`; the move `R` increases `x` by 1, `L` decreases `x`, `U` increases `y`, and `D` decreases `y`. Print how many distinct squares the robot visited (including the starting square) and whether it visited any square more than once.

16. Create the set `frozen_primes = frozenset({2, 3, 5, 7})`. Check whether it is possible to add an element to it. Then create a set of sets whose elements are `frozenset` objects, and explain in a comment why the elements of a set cannot be ordinary sets or lists.

17. Read a natural number `n` from the user and determine the set of prime numbers less than `n` using the Sieve of Eratosthenes, storing the composite numbers in the set `composite`. Print the result as a sorted list.

18. For `n = 100 000`, create a list and a set containing the numbers from 0 to `n - 1`. Using the `time` module (`time.perf_counter()`), measure the total execution time of 1000 membership tests (`in`) for the number `n - 1` in the list and in the set. Print the results and explain the difference in a comment.

8 Dictionaries

1. Create a dictionary `student` with the keys `name`, `age` and `grade`. Assign any values to them, then print the value assigned to the key `name`.
2. Given the dictionary `fruits = {"apple": 3, "banana": 5, "cherry": 2}`, add a new key `orange` with the value 7, then print the whole dictionary.
3. Given the dictionary `config = {"host": "localhost", "port": 8080, "debug": True}`, remove the key `debug` using the `pop()` method and print the removed value as well as the remaining dictionary.
4. Given the dictionary `prices = {"milk": 3.50, "bread": 2.00, "eggs": 5.99, "butter": 4.25}`, write a loop that prints only the products whose price is higher than 3.00.
5. Use a dict comprehension to create a dictionary `squares` that maps each number from 1 to 10 to its square (e.g. `{1: 1, 2: 4, 3: 9, ...}`).
6. Given two lists, `keys = ["name", "city", "language"]` and `values = ["Alice", "Warsaw", "Python"]`, create a dictionary `profile` from them using the `zip()` function.
7. Given the dictionary `inventory = {"apples": 10, "bananas": 0, "cherries": 5, "dates": 0}`, use a dict comprehension to create a new dictionary `in_stock` that contains only the products with a quantity greater than 0.
8. Given the dictionary `scores = {"Alice": 85, "Bob": 92, "Charlie": 78, "Diana": 95}`, find the key with the highest value using the `max()` function and its `key` parameter.
9. Write a function `merge_dicts(dict_a, dict_b)` that merges two dictionaries. If both dictionaries contain the same key, the values should be added together. Return a new dictionary.
10. Given the text `text = "apple banana apple cherry banana apple"`, write code that counts the occurrences of each word and stores the result in a dictionary `word_count` without using the `Counter` class.
11. Given the dictionary `data = {"a": 1, "b": 2, "c": 3}`, write code that swaps the keys and values, creating a new dictionary `inverted_data` (e.g. `{1: "a", 2: "b", 3: "c"}`).

12. Create a nested dictionary `school` that stores data about three students. Each student (with the keys `student_1`, `student_2`, `student_3`) is assigned a dictionary with the keys `name`, `age` and `grades` (a list of grades). Print the average grade of the second student.

13. Write a function `group_by_length(words)` that takes a list of strings and returns a dictionary in which the key is the length of a word and the value is a list of words of that length. Use the `setdefault()` method.

14. Given the dictionary `settings = {"resolution": "1080p", "volume": 75, "brightness": 50}`, write a function `update_settings(current, overrides)` that updates the values in `current` only for keys that already exist in it, ignoring any new keys from `overrides`.

15. Given the dictionary `nested = {"a": {"b": {"c": 42}}}`, write a function `deep_get(dictionary, keys_path, default_value=None)` that takes a dictionary and a string of keys separated by dots (e.g. `"a.b.c"`) and returns the nested value, or `default_value` if the path does not exist.

16. Write a function `flatten_dict(nested_dict, parent_key="", separator=".")` that flattens a nested dictionary into a single-level one. For example, `{"a": {"b": 1, "c": {"d": 2}}}` should produce `{"a.b": 1, "a.c.d": 2}`. Use recursion.

17. Write a function `diff_dicts(dict_a, dict_b)` that compares two dictionaries and returns a dictionary with three keys: `added` (keys present only in `dict_b`), `removed` (keys present only in `dict_a`) and `changed` (keys present in both but with different values, stored as a tuple `(old_value, new_value)`).

9 Functions

1. Write a function `print_header()` that takes no arguments and prints a frame containing the text `Introduction to Programming`. Call it twice.
2. Write a function `square(x)` that returns the square of the number `x`. Call it for several values and print the results. Then use it to compute the sum of the squares of the numbers from 1 to 10.
3. Write a function `say_hello(name)` that only prints a greeting (without a `return` statement). Assign the result of calling it to a variable `result` and print that variable. In a comment, explain where the value `None` came from.
4. Write a function `is_even(n)` that returns `True` if the number is even and `False` otherwise. Write the function body in a single line, returning the result of the comparison directly. Use this function in a loop to print the even numbers from 1 to 20.
5. Write a function `max_of_three(a, b, c)` that returns the largest of three numbers without using the `max()` function. Add a docstring describing what the function does and display it with `help(max_of_three)`.
6. Write a function `is_prime(n)` that checks whether a number is prime. Exit the function with a `return` statement as soon as a divisor is found. Use it to print all prime numbers less than 100.
7. Write a function `min_max_avg(numbers)` that takes a list of numbers and returns the minimum, maximum and average at once (as a tuple). Call it and unpack the result into three variables.
8. Write a function `count_vowels(text)` that returns the number of vowels in a string, and a function `count_consonants(text)` that returns the number of consonants (consider letters only). Test both functions on several strings.
9. Write a function `power(base, exponent=2)` in which the exponent has a default value. Call it with one and with two arguments.
10. Write a function `format_price(amount, currency="PLN", decimals=2)` that returns a string with the amount rounded to the given number of decimal places, followed by the currency symbol. Call it in several ways, making use of the default values.

11. Write a function `add_item(item, items=[])` that adds an element to a list and returns the list. Call it three times without passing the second argument and print the results. Explain in a comment why the result is surprising, then fix the function by using `None` as the default value.

12. Analyze the code below, predict what will be printed, then run it and check your answer:

```
x = 10

def show():
    x = 5
    print("inside:", x)

show()
print("outside:", x)
```

In a comment, explain the concepts of local and global variables.

13. Check what happens when you run the code below and explain the cause of the error:

```
counter = 0

def increment():
    counter = counter + 1

increment()
```

Fix the code in two ways: using the `global` keyword and without it (the function takes the value as an argument and returns the new one). In a comment, indicate which solution is better and why.

14. Create a global dictionary `settings = {"theme": "light", "language": "pl"}` and a function `set_option(key, value)` that changes a value in it. Explain in a comment why the `global` keyword is not needed in this case, while it was needed in task 13.

15. Write a function `print_report(scores)` that takes a dictionary `{name: points}` and prints a report. Inside it, define a nested function `grade(points)` that returns a grade based on the number of points (any scale), and use it when printing each line of the report. Check whether the `grade` function can be called from outside `print_report`.

16. Write a function `make_multiplier(factor)` that returns a nested function multiplying its argument by `factor`. Use it to create the functions `double` and `triple` and test them.

17. Write a function `make_counter()` that returns a nested function which, on each call, increases an internal counter by 1 and returns its current value. Use the `nonlocal` keyword. Show that two counters created by `make_counter()` work independently.

18. Write a recursive function `factorial(n)` that computes the factorial, as well as its iterative version `factorial_iter(n)`. Compare the results for `n` from 0 to 10.

19. Write a recursive function `fibonacci(n)` that returns the `n`-th term of the Fibonacci sequence. Add a global call counter and check how many times the function is called for `n = 10`, `n = 20` and `n = 25`. In a comment, explain where such a large number of calls comes from.

20. Write a recursive function `sum_digits(n)` that returns the sum of the digits of a natural number, and a recursive function `is_palindrome(text)` that checks whether a string is a palindrome (by comparing the first and last characters and calling itself on the string without them).

21. Write a program for managing a to-do list, divided into functions: `add_task(tasks, title)`, `remove_task(tasks, index)`, `show_tasks(tasks)` and `main()`, which displays a menu in a loop, reads the user's choice and calls the appropriate functions. The task list should be created in the `main()` function and passed to the other functions as an argument — do not use global variables.

10 Function Arguments

1. Write a function `add_numbers(a, b)` that takes two positional arguments of type int and returns their sum. Call it with the arguments 3 and 5.
2. Write a function `greet(name, greeting="Hello")` that takes a name as a required argument and a greeting as an argument with a default value. The function should return a formatted greeting text. Call it in two ways — with one and with two arguments.
3. Write a function `describe_pet(animal_type, pet_name)` and call it, passing both arguments as keyword arguments in the reverse order to the one in the function definition.
4. Write a function `make_sandwich(*ingredients)` that takes any number of positional arguments representing sandwich ingredients and returns them as a single list. Call it with three and then with six arguments.
5. Write a function `build_profile(**user_info)` that takes any number of keyword arguments and returns them as a dictionary. Call it, passing at least four different pieces of information about a user.
6. Write a function `calculate_power(base, exponent=2)` in which the argument `exponent` has a default value of 2. Call it three times — without specifying the exponent, with an exponent of 3, and with the exponent passed as a keyword argument.
7. Write a function `format_address(street, city, zip_code, country="Poland")` that combines address data into a single formatted string. Call it, mixing positional and keyword arguments in a single call.
8. Write a function `apply_discount(price, discount_percent, /, tax_rate=0.23)` that uses the `/` operator to force `price` and `discount_percent` to be positional-only arguments. Show that an attempt to pass them as keyword arguments results in an error.
9. Write a function `register_user(username, *, email, age)` that uses the `*` operator to force `email` and `age` to be passed only as keyword arguments. Call it correctly, then show an incorrect call using positional arguments only.
10. Write a function `mix_paint(primary, secondary, /, *, finish="matte")` that combines both mechanisms — positional-only arguments (before `/`) and keyword-only arguments (after `*`). Call it in three different ways.

11. Write a function `log_message(message, *tags, level="INFO")` that takes one required argument, any number of tags via `*args`, and a keyword argument with a default value at the end. Call it with two tags and a changed logging level.

12. Write a function `create_report(title, *sections, **metadata)` that combines a required argument, `*args` and `**kwargs` in a single signature. Call it, passing a title, three sections and two key-value pairs as metadata.

13. Write two functions — `square(n)`, which returns the square of a number, and `double(n)`, which returns the number doubled. Then write a function `apply_operation(func, value)` that takes another function as an argument and calls it on the given value. Test it with both functions.

14. Write a function `process_data(data, *, transformer=None, validator=None)` in which both optional arguments are functions (or None). If `transformer` is not None, the function should transform the data, and if `validator` is not None, it should validate the result. Call it with a lambda function as the transformer.

15. Write a function `repeat_action(action, times=1, /, *, delay=0)`, then call it, passing a lambda function that prints some text as the `action` argument. Show three calls with different combinations of positional and keyword arguments.

16. Write a function `merge_configs(*config_dicts, **overrides)` that takes any number of dictionaries as `*args`, merges them into one, and then overrides the values passed as `**kwargs`. Test it with three configuration dictionaries and two overrides.

17. Write a function `safe_divide(numerator, denominator, /, *, on_error="return_none")` that divides two numbers and, in the case of division by zero, behaves according to the value of the `on_error` argument — it can return None, return zero, or raise an exception. Test each of the three modes.

18. Write a function `pipeline(value, *transformations)` that takes an initial value and any number of functions. The function should apply each of them in turn to the result of the previous one. Test it by passing three lambda functions that form a chain of transformations.

19. Write a decorator `validate_types(*expected_types)` that takes, via `*args`, the expected types of the successive arguments of the decorated function. Before calling the function, the decorator should check whether the types of the passed arguments match the expected ones and raise a TypeError if they do not. Apply it to a function `concat_strings(a, b)`.

20. Write a function `create_dispatcher()` that returns two inner functions — `register(name, func)` for registering a function under a given name, and `call(name, *args, **kwargs)` for calling a registered function with arbitrary arguments. Test the mechanism by registering three functions with different signatures and calling them

through the dispatcher with various combinations of positional, keyword and default arguments.

11 f-strings

1. Define a variable `name = "Anna"` and a variable `age = 28`, then print them in a single sentence using an f-string of the form `f"Name: {name}, age: {age}"`.
2. Given the variable `age = 28`, write an f-string that evaluates the expression `age + 5` inside the braces and prints the result in the sentence `"In 5 years you will be: ..."`.
3. Define a variable `name = "Anna"` and write an f-string that calls the `name.upper()` method inside the braces, printing the name in uppercase letters.
4. Given the variable `pi = 3.141592653589793`, write an f-string with the `:.4f` specifier that displays the value of pi rounded to four decimal places.
5. Define a variable `loss = 0.00034521` and use an f-string with the `:.2e` specifier to print this value in scientific notation with two decimal places.
6. Given the variable `accuracy = 0.9534`, write an f-string with the `:.1%` specifier that displays the accuracy as a percentage with one decimal place.
7. Define a variable `population = 38179800` and write an f-string with the `:,` thousands separator, printing the value in a readable format with commas.
8. Given the variable `revenue = 1543267.89`, write an f-string that combines the thousands separator with a precision of two decimal places using the `:,.2f` specifier, and appends the currency `PLN` at the end.
9. Define a variable `population = 38179800` and use an f-string with the `:_` specifier to print the value with underscores as thousands separators.
10. Given the list of tuples `metrics = [("Accuracy", 0.9534), ("Precision", 0.9211), ("Recall", 0.8876)]`, write a `for` loop that uses an f-string with left alignment `:<12` to print the metric names in a 12-character-wide column, followed by their values in the `:.2%` format.
11. Using the same `metrics` list, write a `for` loop with an f-string in which you apply right alignment `:>15` with an asterisk `*` as the fill character, followed by a colon and the value in the `:.4f` format.
12. Given the variable `x = 42`, write an f-string with the `:^10` specifier that centers the number in a 10-character-wide field. Describe the role of the `^` operator here.

13. Import `datetime` and define `now = datetime.now()`. Write an f-string with the `:%Y-%m-%d` format that prints the current date in the ISO standard.

14. Using the same variable `now`, write two f-strings: one with the `:%H:%M:%S` format that prints the time, and another with the `:%d %B %Y` format that prints the date in the European format with the month name spelled out.

15. Define the variables `learning_rate = 0.001`, `epochs = 50` and `batch_size = 32`. Use the `=` debugging operator in an f-string (e.g. `f"{learning_rate = }"`) to print the name and value of each variable.

16. Given the variable `loss = 0.0023456`, write an f-string that combines the `=` operator with formatting, in the form `f"{loss = :.4f}"`, and describe what will be printed.

17. Given the variable `train_acc = 0.9812`, write an f-string that uses the `=` operator and the `:.1%` specifier to print both the variable name and its percentage value at the same time.

18. Define a list `values = [10, 20, 30, 40]` and write an f-string in which you evaluate the expression `sum(values) / len(values)` inside the braces, printing the arithmetic mean.

19. Given the variables `model_name = "RandomForestClassifier"`, `accuracy = 0.9534`, `precision = 0.9211`, `recall = 0.8876` and `n_samples = 150000`, write a multi-line f-string (in triple quotes) that formats a report of the model's results. Use lines with `{'='*50}` as separators, right alignment `:>8.2%` for the metrics, and the `:,` thousands separator for the number of samples.

20. Using the variables from the previous task, additionally compute `f1 = 2 * precision * recall / (precision + recall)` and extend the multi-line f-string with a row for the F1-score. Make sure that all metrics are uniformly right-aligned in an 8-character-wide column with the `:.2%` specifier, and that the whole report is surrounded by decorative lines generated by an expression in braces.

12 Exception Handling

1. Run the following statements one at a time (each separately) and write the name of the raised exception in a comment:

```
10 / 0
int("abc")
[1, 2, 3][5]
{"a": 1}["b"]
"5" + 5
undefined_variable
```

2. Read an integer from the user. If the user enters a value that cannot be converted, catch the `ValueError` exception and print a clear message instead of the interpreter's error message.
3. Read two numbers from the user and print the result of dividing them. Handle the `ValueError` (invalid input) and `ZeroDivisionError` (division by zero) exceptions in separate `except` blocks, printing a different message for each.
4. Modify the program from the previous task so that the result of the division is printed in the `else` block, and the message `"End of calculations"` is printed in the `finally` block. Check when the `else` block is executed and when the `finally` block is executed.
5. Catch an exception using the construct `except ValueError as error:` and print the message text (`error`) and the exception type (`type(error).__name__`).
6. Write a function `read_int(prompt)` that keeps reading input from the user in a loop until they enter a valid integer, and returns that number. Use it to read the user's age.
7. Write a function `read_int_in_range(prompt, low, high)` that works like the function from the previous task, but additionally requires the number to be within the range `[low, high]`. Test it by reading a grade in the range from 2 to 5.
8. Given the list:

```
data = ["12", "7", "abc", "3.5", "", "40", "-8"]
```

Using a loop and a `try/except` block, create a list `numbers` containing only the elements that can be converted to `int`, and a list `errors` containing the remaining elements.

9. Given the dictionary:

```
stock = {"apple": 10, "banana": 0, "cherry": 25}
```

Write a function `get_stock(product)` that returns the number of items of a product and, if the key does not exist, catches `KeyError` and returns `0`. Then write a second version of the function without exception handling, using the `.get()` method. In a comment, compare both approaches.

10. Write a function `safe_get(items, index, default=None)` that returns the list element at the given index and, in the case of an `IndexError` exception, returns the default value.

11. Write a function `calculate_average(numbers)` that raises a `ValueError` exception with the message `"The list cannot be empty"` (the `raise` statement) when an empty list is passed, and a `TypeError` when any element is not a number (use `isinstance(x, (int, float))`). Test the function in a `try/except` block.

12. Write a function `set_age(age)` that raises a `ValueError` if the age is negative or greater than 150, and otherwise returns it. Test it in a loop for the list of values `[25, -3, 200, 47]`, printing either the result or the error message for each value.

13. Handle several exceptions in a single block: `except (ValueError, ZeroDivisionError) as error:`. Write a program that reads two numbers and prints the result of dividing them, and in the case of either of these two errors prints one common message containing the exception text.

14. Explain in a comment why the construct `except:` (without an exception name) or `except Exception:` covering a large portion of code is considered bad practice. Show an example in which such a block hides a programmer's mistake (e.g. a typo in a variable name).

15. Write a function `parse_point(text)` that takes a string of the form `"x,y"` (e.g. `"3,4"`) and returns a tuple of two floating-point numbers. If the string has an invalid format, the function should catch the exception and then raise a new `ValueError` with a clear message, chaining it to the original exception using `raise ... from error`.

16. Given the function:

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount
```

Write a program that, in a loop, reads withdrawal amounts for an initial balance of 1000 PLN and calls the `withdraw` function. The program handles errors both during data conversion and those raised by the function, and terminates when `q` is entered.

17. Write a function `divide(a, b)` in which you first use an `assert` statement with an error message to check that `b != 0`. Call it with invalid data and check which exception is raised. In a comment, explain why `assert` should not replace validation of user input.

18. Analyze the code below and predict the order of the printed messages for the calls `test(1)`, `test(0)` and `test("a")`. Check your answer by running the code.

```
def test(x):
    try:
        print("try")
        result = 10 / x
    except ZeroDivisionError:
        print("except ZeroDivisionError")
        return "error"
    else:
        print("else")
        return result
    finally:
        print("finally")
```

What happens to the `TypeError` exception in the call `test("a")`?

19. Write a calculator for expressions of the form `number operator number` (e.g. `12 * 4`), read from the user in a loop until `end` is entered. Use the `.split()` method. Handle the following exceptions: a wrong number of elements in the expression, invalid numbers, division by zero and an unknown operator — each with an appropriate message. The program must not terminate with an error regardless of the data entered.

13 File Handling

In these tasks, open files using the `with` statement, and for text files specify the encoding explicitly (`encoding="utf-8"`).

13.1 Text Files

1. Create a file `hello.txt` and write three lines of text to it (mode `"w"`, method `.write()`). Remember the newline characters `"\n"`. Then open the file in mode `"r"`, read its entire contents with the `.read()` method and print them.
2. Open the file `hello.txt` from the previous task in mode `"a"` and append two new lines at its end. Print the contents of the file and check whether the previous lines have been preserved. In a comment, describe the difference between the modes `"w"`, `"a"` and `"x"`.
3. Read the file `hello.txt` line by line by iterating directly over the file object, and print each line preceded by its number. Remove the newline characters with the `.rstrip("\n")` method to avoid empty lines in the output.
4. Compare how the `.read()`, `.readline()` and `.readlines()` methods work on the same file. Check the type of the returned result in each case.
5. Read successive names from the user (until an empty string is entered) and save them in the file `names.txt` — each on a separate line — using the `.writelines()` method.
6. Write a function `file_stats(path)` that returns a tuple with the number of lines, the number of words and the number of characters in a text file (similar to the `wc` command on Unix systems). Test it on a file of your choice.
7. Write a program that tries to open a file whose name is given by the user and prints its contents. If the file does not exist, catch the `FileNotFoundError` exception and print an appropriate message.
8. Create a file `numbers.txt` containing integers on successive lines (some lines may be empty or contain text that is not a number). Write a program that reads the file, skips invalid lines (handling the `ValueError` exception) and prints the sum, minimum, maximum and the number of skipped lines.

9. Write a function `count_words(path)` that returns a dictionary with the number of occurrences of each word in a text file. Convert all letters to lowercase and remove the punctuation characters `.,;:!?`. Print the 10 most frequent words (hint: `sorted(counts, key=counts.get, reverse=True)`).

10. Write a function `copy_uppercase(source, target)` that copies the text file `source` to the file `target`, converting all letters to uppercase. Open both files in a single `with` statement.

11. Given the file `grades.csv` with the following contents:

```
name;math;physics;computer_science
Alice;5;4;5
Bob;3;4;4
Charlie;4;5;3
Diana;5;5;5
```

Read the file without using any additional modules (the `.strip()` and `.split(";")` methods), skip the header row and print the average grade for each person. Then save the names and averages (rounded to two decimal places) to the file `averages.csv`.

12. Solve the previous task again, this time using the `csv` module (`csv.reader` with the parameter `delimiter=";"` and `csv.writer`). When opening the file for writing, use the parameter `newline=""`.

13. Write a function `save_contacts(contacts, path)` that saves a dictionary of contacts `{name: phone number}` to a text file in the form `name=number` (each contact on a separate line), and a function `load_contacts(path)` that reads such a file and returns a dictionary. Check that after saving and loading you get a dictionary equal to the original one.

14. Save the same dictionary of contacts to the file `contacts.json` using the `json.dump()` function (with the parameters `ensure_ascii=False` and `indent=2`) and read it back with the `json.load()` function. Open the file in a text editor and compare its contents with the file from the previous task.

15. Write a journal program: each time it is run, it reads one line of text from the user and appends it to the end of the file `journal.txt` together with the number of the entry (determine the number by counting the existing lines of the file; take into account the case where the file does not exist yet).

13.2 Binary Files

16. Create a `bytes` object using `bytes([72, 101, 108, 108, 111])` and print it. Then encode the string `"Zażółć gęślą jaźń"` (a Polish phrase full of diacritics) into bytes with the `.encode("utf-8")` method, and print the number of characters in the string and the number of bytes after encoding. In a comment, explain the difference.

17. Write the string `"Zażółć gęślą jaźń"` to a text file with `utf-8` encoding, then open this file in binary mode `"rb"` and print its contents as a `bytes` object as well as a list of the values of successive bytes (`list(data)`). Also try reading the file in text mode with the `cp1250` encoding (a legacy Central European Windows code page) and describe the result.

18. Write the numbers from 0 to 255 as successive bytes to the binary file `data.bin` (mode `"wb"`). Read the file, check its size (`len()`) and print the bytes at indices 65 to 90. Decode them into a string and explain the result.

19. Write the integers `[1, 300, 70000, -5]` to a binary file, converting each of them into 4 bytes with the `int.to_bytes(4, "little", signed=True)` method. Then read the file 4 bytes at a time (`.read(4)`) and reconstruct the numbers with the `int.from_bytes()` method.

20. Write a function `copy_binary(source, target, chunk_size=4096)` that copies any file (e.g. a JPG image) in binary mode, reading it in chunks of size `chunk_size` in a `while` loop. Check whether the copy is identical to the original by comparing the contents of both files read in binary mode.

21. PNG files start with the 8-byte signature `b"\x89PNG\r\n\x1a\n"`, and JPEG files start with the bytes `b"\xff\xd8\xff"`. Write a function `detect_image_type(path)` that reads the beginning of a file and returns `"PNG"`, `"JPEG"` or `"unknown"`. Test it on several files, including a text file.

22. Open the file `data.bin` from task 18 in mode `"rb"`. Using the `.seek()` method, move to the byte at index 100, read 5 bytes and print them. Then print the current position in the file using the `.tell()` method.

14 Program Structure and Modules

1. Import the `math` module and print the values of the constants `math.pi` and `math.e`, as well as the results of the functions `math.sqrt(2)`, `math.floor(3.7)`, `math.ceil(3.2)` and `math.gcd(48, 18)`.

2. Import only the functions `sqrt` and `hypot` from the `math` module (`from math import sqrt, hypot`) and compute the length of the diagonal of a rectangle with sides 3 and 4 in two ways. In a comment, describe the difference between `import math` and `from math import sqrt`.

3. Import the `random` module under a shorter name (`import random as rnd`). Generate and print: a random integer in the range 1–6 (`randint`), a random floating-point number in the interval $[0, 1)$ (`random`), a random element of a list (`choice`) and a list shuffled in place (`shuffle`). Set the generator seed (`seed`) and check that the results become reproducible.

4. Use the `dir()` function to print the names available in the `math` module. Then use the `help()` function to display the documentation of the `math.isclose` function and, based on it, compare the values `0.1 + 0.2` and `0.3`.

5. Using the `random` module, write the number-guessing game from the Loops chapter, in which the secret number is drawn at random each time the program is run.

6. Create a file `geometry.py` containing the functions `circle_area(r)`, `circle_perimeter(r)`, `rectangle_area(a, b)` and the constant `PI = 3.14159`. In the same directory, create a file `main.py` that imports the `geometry` module and uses its functions. Run `main.py`.

7. In the `main.py` file from the previous task, import selected functions from the `geometry` module in three ways: `import geometry`, `import geometry as geo`, `from geometry import circle_area`. In a comment, explain why `from geometry import *` is discouraged.

8. At the end of the `geometry.py` file, add the statement `print("Module geometry loaded")`. Run `main.py` and check when the message appears. Import the module twice and check how many times the message is printed.

9. In the `geometry.py` module, print the value of the `__name__` variable. Check what its value is when you run the file directly and when it is imported from `main.py`. Then move the code that tests the module's functions into an `if __name__ == "__main__":` block so that it is not executed on import.

10. Restructure the program below so that all the code is inside functions and the program is started by a `main()` function called in an `if __name__ == "__main__":` block:

```
numbers = []
while True:
    text = input("Enter a number (empty = finish): ")
    if text == "":
        break
    numbers.append(float(text))
print("Sum:", sum(numbers))
print("Average:", sum(numbers) / len(numbers))
```

Extract separate functions for reading the data and computing the statistics. Make sure to handle an empty list.

11. Write a module `text_tools.py` with the functions `count_words(text)`, `reverse_words(text)` and `is_palindrome(text)` (ignoring letter case and spaces). Add a docstring at the beginning of the module and for each function. In a separate file, import the module and call `help(text_tools)`.

12. Create a package `shapes` — a directory with an `__init__.py` file and the modules `circle.py` and `square.py`, each of which contains the functions `area()` and `perimeter()`. In a `main.py` file (outside the package directory), import the modules in two ways: `from shapes import circle` and `import shapes.square`.

13. In the `shapes/__init__.py` file from the previous task, import the `circle` and `square` modules (a relative import: `from . import circle, square`), so that in `main.py` it is enough to write `import shapes` and use `shapes.circle.area(...)`.

14. Import the `sys` module and print the `sys.path` list. In a comment, explain in what order Python searches for imported modules and what happens when you create your own file named `random.py` in the project directory.

15. Write a program `sum_args.py` that sums the numbers passed as command-line arguments (`sys.argv`). Run it with the command `python sum_args.py 3 4 5.5`. Handle the case of no arguments and of arguments that are not numbers.

16. Split the program from the Functions chapter (the to-do list) into modules:

    - `storage.py` — functions for saving the task list to and loading it from a text file,
    - `tasks.py` — functions for adding, removing and displaying tasks,
    - `main.py` — the program menu and the `main()` function.

    Make sure that the `storage.py` and `tasks.py` modules do not use the `input()` or `print()` functions (except for the function that displays the tasks) — all communication with the user should take place in `main.py`.

17. Create a module `config.py` containing configuration constants (e.g. `FILE_NAME`, `MAX_TASKS`) written in uppercase letters, and use them in the program from the previous task instead of values hard-coded directly in the code.

15 Clean Code Principles

The exercises in this chapter concern the recommendations of PEP 8 and tools that support writing clean code. Tools are installed with the command `pip install <name>` (preferably in a virtual environment).

1. Fix the names in the code below according to PEP 8 conventions (`snake_case` for variables and functions, `UPPER_CASE` for constants). Give the names a meaning that explains the role of each variable.

```
def F(L):
    S = 0
    for X in L:
        S += X
    return S / len(L)

A = [3, 5, 4]
b = F(A)
print(b)
```

2. Fix the formatting of the code below according to PEP 8 (indentation, spaces around operators and after commas, blank lines, line length):

```
import math,random
def area(r):return math.pi*r**2
def main():
  for i in range(5):
    r=random.randint(1,10)
    print("r =",r,"area =",round(area(r),2))
main()
```

3. Install the `ruff` tool and run `ruff check` on the file from the previous exercise (the version before your fixes). Analyze the reported problems, then run `ruff format` and compare the result with your own fixes.

4. Install the `pylint` tool and run it on a chosen solution from the Functions chapter. Improve the code so that it receives a score of at least 9/10. Write down in a comment which kinds of warnings appeared most often.

5. Replace the “magic numbers” in the code below with constants that have readable names:

```
def final_price(price):
    if price > 500:
        price = price * 0.9
    return price * 1.23
```

6. Add docstrings (compliant with PEP 257) to the module and to every function from exercise 5. Describe the arguments, the return value and the raised exceptions. Display them using `help()`.

7. Add type annotations (type hints) to the functions:

```
def average(numbers):
    return sum(numbers) / len(numbers)

def find_user(users, name):
    for user in users:
        if user["name"] == name:
            return user
    return None
```

Install the `mypy` tool and use it to check code in which the `average` function is called with a string instead of a list.

8. Simplify the code below by using an early return (`return`) instead of nested conditional statements:

```
def can_vote(age, is_citizen, is_registered):
    if age >= 18:
        if is_citizen:
            if is_registered:
                return True
            else:
                return False
        else:
            return False
    else:
        return False
```

Then write the function in a single line as a `return` with a logical expression.

9. Simplify the following code fragments using Python idioms:

```
if is_valid == True:
    print("OK")

if len(items) == 0:
    print("Empty")

for i in range(len(names)):
    print(i, names[i])

result = []
for x in range(10):
    if x % 2 == 0:
        result.append(x * x)
```

10. In the code below, the same fragment is repeated three times. Apply the DRY principle (*Don't Repeat Yourself*) by extracting a function:

```
math_scores = [4, 5, 3, 5]
print("Mathematics:", round(sum(math_scores) / len(math_scores), 2))
physics_scores = [3, 4, 4]
print("Physics:", round(sum(physics_scores) / len(physics_scores), 2))
cs_scores = [5, 5, 4, 5]
print("Computer Science:", round(sum(cs_scores) / len(cs_scores), 2))
```

11. The function below does too many things at once. Split it into smaller functions, each of which has a single responsibility (reading data, performing calculations, printing results):

```
def process():
    numbers = []
    while True:
        text = input("Number: ")
        if text == "":
            break
        numbers.append(int(text))
    even = [n for n in numbers if n % 2 == 0]
    odd = [n for n in numbers if n % 2 != 0]
    print("Even:", even, "sum:", sum(even))
    print("Odd:", odd, "sum:", sum(odd))
```

12. Add usage examples to the docstring of the `is_prime(n)` function from the Functions chapter in the format of the `doctest` module (lines starting with `>>>`). Run the tests with the command `python -m doctest -v file_name.py`.

13. Write a file `test_geometry.py` with tests for the functions of the `geometry.py` module from the Program Structure and Modules chapter, using `assert` statements in functions whose names start with `test_`. Install the `pytest` tool and run the tests with the command `pytest`. Deliberately introduce a bug into one of the functions and check how `pytest` reports the failure.

14. Analyze the code below and list in a comment at least five problems related to the readability and quality of the code. Then rewrite it in a clean form.

```
def calc(d):
    # d - data
    r = {}
    for k in d:
        t = 0
        for i in d[k]:
            t = t + i
        r[k] = t / len(d[k]) if len(d[k]) != 0 else 0
    l = []
    for k in r:
        if r[k] >= 4.5: l.append(k)
    return r, l
```

15. Create a `pyproject.toml` file in the project directory with a configuration for the `ruff` tool (e.g. a maximum line length of 88 characters, enabled rules `E`, `F`, `I`, `N`). Run `ruff check` on the whole project from the previous chapters and fix the reported problems.

Part II

Object-Oriented Programming

16 Simple Classes

For each class, create an object of it outside the class at least once.

1. Write a class `Dog` with an initializer that takes `name` and `age`, and a method `bark()` that prints `"<name>: Woof!"`. Create two objects of this class and call the `bark()` method on each of them. Also print the attributes of both objects.

2. Write a class `Rectangle` with attributes `width` and `height` and methods `area()` and `perimeter()` that return the area and the perimeter of the rectangle, respectively. Add a method `is_square()` that returns a boolean value.

3. Write a class `Counter` with an attribute `value` (initially 0) and methods `increment()`, `decrement()` and `reset()`. Create two counters, perform various operations on them and show that their states are independent.

4. Write a class `Lamp` with an attribute `is_on` (initially `False`) and methods `turn_on()`, `turn_off()` and `toggle()`. Add a method `status()` that returns the string `"on"` or `"off"`.

5. Write a class `BankAccount` with attributes `owner` and `balance` (0 by default) and methods `deposit(amount)` and `withdraw(amount)`. The methods should raise a `ValueError` exception when the amount is not positive or when the account has insufficient funds. Test the class by catching the exceptions in a `try/except` block.

6. Add to the `BankAccount` class from the previous exercise an attribute `history` (a list) that records each operation as a tuple `(operation_type, amount)`, and a method `print_history()` that prints the operation history.

7. Add to the `BankAccount` class a method `transfer(other, amount)` that transfers money to another account by calling the `withdraw()` and `deposit()` methods. Create two accounts and make a transfer. Check what happens to the balances when the transfer fails.

8. Analyze the code below and explain in a comment why changing an attribute of object `b` also changes object `a`. Then create a second, independent object.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

a = Point(1, 2)
b = a
b.x = 100
print(a.x)
```

9. Write a class `Point` with attributes `x` and `y` and methods `move(dx, dy)` (moves the point in place) and `distance_to(other)` (returns the distance to another point). Create three points and print the distances between each pair.

10. Write a class `Student` with attributes `name` and `grades` (an empty list created in the initializer) and methods `add_grade(grade)` (which checks whether the grade belongs to the set `{2, 3, 3.5, 4, 4.5, 5}`), `average()` and `has_passed()` (returns `True` if no grade is equal to 2).

11. Create a list of five objects of the `Student` class from the previous exercise, add a few grades to each of them, and then:

    - print the name and the average of each student,
    - find the student with the highest average,
    - create a list of the names of the students who failed the course.

12. Write a class `Car` with a class attribute `wheels = 4` and instance attributes `brand` and `model`. Print the `wheels` attribute through the class and through an object. Then assign `car.wheels = 3` for one object and check the value of the attribute in other objects and in the class. Explain the result in a comment.

13. Write a class `Ticket` with a class attribute `counter` that counts the number of tickets created. Each newly created ticket should receive the next number as the instance attribute `number`. Create several tickets and print their numbers and the total number of tickets.

14. Write a class `ShoppingCart` with methods `add(product, price, quantity=1)`, `remove(product)`, `total()` and `print_receipt()`. Store the products in a dictionary in which the key is the product name and the value is a tuple `(price, quantity)`. Adding an existing product should increase its quantity.

15. Write a class `Stack` that stores elements in a list, with methods `push(item)`, `pop()`, `peek()`, `is_empty()` and `size()`. The `pop()` and `peek()` methods called on an empty stack should raise an `IndexError` exception with a readable message. Use the stack to check whether the brackets in a string (e.g. `"(a[b]{c})"`) are correctly paired.

16. Write a class `Library` that stores books as a dictionary `{title: number_of_copies}`, with methods `add_book(title, copies=1)`, `borrow(title)` and `give_back(title)`. The `borrow` method should raise an exception when the book is not in the library or all copies are borrowed. Add methods `save(path)` and `load(path)` that save and load the state of the library to and from a text file.

17 Initializer

For each class, create an object of it outside the class at least once.

1. Write a class `Book` with an initializer that takes two arguments — `title` and `author`. Both should be stored as instance attributes. Create two objects of this class and print their attributes.
2. Write a class `Circle` with an initializer that takes an argument `radius`. Add an instance attribute `area` that is calculated automatically when the object is created, based on the given radius.
3. Write a class `Student` with an initializer that takes `name` and an optional argument `grade` with the default value "A". Create one student with the default grade and another one with an explicitly given grade.
4. Write a class `Color` with an initializer that takes three arguments — `red`, `green` and `blue`, each with the default value 0. Create an object by passing the arguments exclusively as keyword arguments.
5. Write a class `ShoppingList` with an initializer that takes `owner` as a required argument. The initializer should additionally create an empty attribute `items`, which is a list. Do not pass this list as an argument.
6. Write a class `Config` with an initializer that takes `app_name` as a required argument. The initializer should create an empty attribute `settings`, which is a dictionary. Then manually add two key-value pairs to this dictionary on the created object.
7. Write a class `Playlist` with an initializer that takes `name` and an optional argument `songs`, which should default to an empty list. Correctly solve the mutable default argument problem by using the value None and a conditional assignment inside the initializer.
8. Write a class `TemperatureReading` with an initializer that takes `value` and `unit` with the default value "Celsius". Add validation — if `unit` is not one of the strings "Celsius", "Fahrenheit" or "Kelvin", the initializer should raise a ValueError with an appropriate message.
9. Write a class `User` with an initializer that takes `username` and `email`. Add validation that checks whether `email` contains the "@" character — if not, the initializer should raise a ValueError. Test creating an object with a valid and an invalid address.

10. Write a class `BankAccount` with an initializer that takes `owner` and an optional `balance` with the default value 0. Add validation — if `balance` is a negative number, the initializer should raise a ValueError. Create an account with a positive balance and test an attempt to create an account with a negative balance.

11. Write a class `Product` with an initializer that takes `name`, `price` and an optional `tags` (a list of tags with the default value None). Add validation — `price` must be a positive number and `name` cannot be an empty string. Solve the mutable default argument problem for `tags`.

12. Write a class `Scoreboard` with an initializer that takes `game_name` and an optional `scores`, which is a dictionary (None by default). Inside the initializer, create a copy of the given dictionary instead of assigning the reference directly. Show why this matters by modifying the original dictionary after the object has been created.

13. Write a class `DateRange` with an initializer that takes `start_date` and `end_date` as strings in the "YYYY-MM-DD" format. The initializer should convert them into `datetime.date` objects and validate that `start_date` is not later than `end_date`.

14. Write a class `Survey` with an initializer that takes `title`, `*questions` (any number of questions as `*args`) and an optional `anonymous` with the default value True. Store the questions as a list in an instance attribute. Create an object with five questions.

15. Write a class `ServerConfig` with an initializer that takes `host` and `port` as positional-only arguments (using `/`) and `timeout` and `max_retries` as keyword-only arguments (using `*`), both with default values. Show a correct and an incorrect call.

16. Write a class `Matrix` with an initializer that takes an argument `data`, which is a list of lists (a two-dimensional matrix). Add validation that checks whether all inner lists have the same length — if not, raise a ValueError. Also store the attributes `rows` and `cols` calculated from the data.

17. Write a class `PasswordPolicy` with an initializer that takes `min_length`, `require_uppercase`, `require_digit` and `require_special_char` — all with reasonable default values. Add validation — `min_length` must be an integer in the range from 6 to 128, and the remaining arguments must be of type bool. Raise a TypeError or a ValueError if the conditions are not met.

18. Write a class `EventLog` with an initializer that takes `source` and an optional `events` (a list of dictionaries, None by default). Each dictionary in the list must contain the keys "timestamp" and "message" — add validation that checks this for every element. The initializer should create a deep copy of the list of dictionaries using `copy.deepcopy`.

19. Write a class `QueryBuilder` with an initializer that takes `table_name` and `**default_filters`. The initializer should validate that `table_name` is a non-empty string consisting only of letters, digits and underscores (protection against SQL

injection). The values from `**default_filters` should be stored as a dictionary in an instance attribute. Create an object with three filters.

20. Write a class `Pipeline` with an initializer that takes `name`, `*steps` and `**options`. Each element of `steps` must be a callable — add validation using the `callable()` function. The options from `**options` should be stored in the `config` attribute as a dictionary. Add validation that checks whether `name` matches the regex pattern `^[a-z][a-z0-9_]*$`. Create an object by passing three lambda functions as steps and two configuration options.

18 Properties

Outside the class, create at least one object and test the properties.

1. Create a class `Circle` with a private attribute `__radius`. Add a property `radius` that returns the value of the radius, and a setter that sets the radius only if the given value is a positive number.
2. Create a class `Person` with a private attribute `__name`. Add a property `name` with a getter that returns the name and a setter that accepts only values of type `str`.
3. Create a class `Temperature` with a private attribute `__celsius`. Add a property `celsius` with a getter and a setter, and a read-only property `fahrenheit` that returns the temperature converted to degrees Fahrenheit.
4. Create a class `Product` with private attributes `__name` and `__price`. Add properties for both attributes. The setter for `price` should accept only values of type `int` or `float` that are greater than or equal to zero.
5. Create a class `Rectangle` with private attributes `__width` and `__height`. Add properties with validation (positive values) and a read-only property `area` that returns the area of the rectangle.
6. Create a class `Student` with a private attribute `__grades`, which is a list. Add a property `grades` whose getter returns a copy of the list and whose setter accepts only lists containing exclusively integers in the range 1–6.
7. Create a class `Email` with a private attribute `__address`. Add a property `address` whose setter validates that the given string contains exactly one `@` character and at least one dot after it.
8. Create a class `BankAccount` with private attributes `__owner` and `__balance`. Add a read-only property `balance` and a property `owner` whose setter accepts only non-empty strings at least 2 characters long.
9. Create a class `Playlist` with a private attribute `__songs` (a list of strings). Add a property `songs` whose getter returns a copy of the list and whose setter accepts only lists in which every element is of type `str` and is not empty.
10. Create a class `Config` with a private attribute `__settings`, which is a dictionary. Add a property `settings` whose getter returns a copy of the dictionary and whose setter accepts only dictionaries in which both the keys and the values are of type `str`.

11. Create a class `Password` with a private attribute `__value`. Add a property `value` whose getter returns a masked form of the password (e.g. `"****"`) and whose setter accepts only strings at least 8 characters long that contain at least one digit and one uppercase letter.

12. Create a class `DateRange` with private attributes `__start` and `__end` (`datetime.date` objects). Add properties for both attributes, where the `end` setter should check that the end date is not earlier than the start date, and the `start` setter — that it is not later than the end date.

13. Create a class `Inventory` with a private attribute `__items`, which is a dictionary mapping product names (str) to quantities (int). Add a property `items` whose getter returns a copy of the dictionary and whose setter validates that all keys are non-empty strings and that the values are non-negative integers.

14. Create a class `RGBColor` with private attributes `__red`, `__green`, `__blue`. Add a property for each channel with a setter that accepts only integers in the range 0–255, and a read-only property `hex` that returns the color in hexadecimal format (e.g. `"#ff00ab"`).

15. Create a class `Matrix` with a private attribute `__data`, which is a list of lists (a 2D matrix). Add a property `data` whose setter checks whether the given value is a non-empty list of lists, whether all rows have the same length and whether all elements are numbers (int or float).

16. Create a class `UserProfile` with private attributes `__username`, `__age` and `__tags` (a list of strings). Add properties for each attribute: `username` accepts only alphanumeric strings 3–20 characters long, `age` accepts integers in the range 0–150, and `tags` accepts lists of unique, non-empty strings with a maximum list length of 10.

17. Create a class `WeightedGrades` with a private attribute `__grades`, which is a dictionary mapping subject names (str) to tuples `(grade, weight)`, where the grade is a float in the range 2.0–5.0 and the weight is a float in the range 0.0–1.0. Add a property `grades` with appropriate validation and a read-only property `weighted_average` that returns the weighted average.

18. Create a class `IPAddress` with a private attribute `__address`. Add a property `address` whose setter validates that the given string is a valid IPv4 address — that is, it consists of exactly four groups of integers in the range 0–255, separated by dots. Add a read-only property `is_private` that returns `True` if the address belongs to a private range (e.g. 192.168.x.x, 10.x.x.x).

19. Create a class `Schedule` with a private attribute `__entries`, which is a dictionary where the key is a day of the week (a str from: "Monday"–"Sunday") and the value is a list of tuples `(start_hour, end_hour)` representing time blocks (int, range 0–23). The setter should validate the correctness of the keys, the tuple format, the hour ranges and that the time blocks within a single day do not overlap. Add a read-only property `total_hours`.

20. Create a class `Document` with private attributes `__title` (str, 1–200 characters), `__authors` (a list of non-empty strings, at least 1 author), `__metadata` (a str→str dictionary) and `__version` (a str in semantic format, e.g. “1.2.3”). Add properties with full validation for each attribute. The `version` setter should additionally check whether the new version is higher than the current one. Add a read-only property `summary` that returns a formatted string containing the title, the authors and the version.

19 Inheritance and Polymorphism

1. Create a class `Animal` with an attribute `name` and a method `speak()` that returns a generic message. Then create a class `Dog` that inherits from `Animal` and overrides the `speak()` method so that it returns a dog's sound.
2. Define a class `Shape` with a method `area()` that returns 0. Create classes `Rectangle` and `Circle` that inherit from `Shape` and implement their own versions of `area()` with the appropriate calculations.
3. Write a class `Vehicle` with attributes `brand` and `max_speed` and a method `describe()`. Create a class `ElectricCar` inheriting from `Vehicle` that adds an attribute `battery_capacity` and extends the `describe()` method with information about the battery.
4. Create a class `Employee` with attributes `name` and `salary` and a method `get_bonus()` that returns 10% of the salary. Create a class `Manager` inheriting from `Employee` in which `get_bonus()` returns 20% of the salary. Demonstrate the use of both classes in a single list and iterate over it.
5. Define a class `Logger` with a method `log(message)`. Create classes `FileLogger` and `ConsoleLogger` that inherit from `Logger` and override `log()` so that they print the message with an appropriate prefix indicating where it is written. Write a function `process(logger)` that takes any object of type `Logger` and calls `log()` on it.
6. Create a class `BankAccount` with methods `deposit(amount)`, `withdraw(amount)` and `get_balance()`. Then create a class `SavingsAccount` that inherits from `BankAccount` but overrides `withdraw()` so that it does not allow a withdrawal if the balance would drop below a set minimum.
7. Write a class `Notification` with a method `send(recipient, text)`. Create three derived classes: `EmailNotification`, `SmsNotification` and `PushNotification`, each of which overrides `send()` and prints a message appropriate for the given channel. Write a function `broadcast(notifications, recipient, text)` that iterates over a list of notifications and sends them polymorphically.
8. Define a class `Converter` with a method `convert(value)` that returns the value unchanged. Create classes `CelsiusToFahrenheit` and `KilometersToMiles` inheriting from `Converter` that override `convert()` with the proper formulas. Use the `isinstance()` function to check the type of an object before conversion.

9. Create a class hierarchy: `Media` (with an attribute `title`), and inheriting from it `Book` (with `author` and `pages`), `Movie` (with `director` and `duration`) and `Podcast` (with `host` and `episodes`). Each class should have a method `info()` that returns a description. Write a function that takes a list of `Media` objects and prints information about each of them.

10. Create a class `PaymentMethod` with a method `pay(amount)`. Create classes `CreditCard`, `PayPal` and `BankTransfer` that inherit from `PaymentMethod`. Each class should override `pay()` and additionally implement a method `validate()` that checks the correctness of the data. In the `CreditCard` class, use `super()` to call the base version of `pay()` before adding its own logic.

11. Write a class `TaskRunner` with methods `setup()`, `execute()` and `teardown()` and a method `run()` that calls these three methods in order. Create classes `DatabaseTask` and `ApiTask` that inherit from `TaskRunner` and override `setup()`, `execute()` and `teardown()` with their own implementations, but do not change the `run()` method — this is an example of the Template Method pattern.

12. Define a class `User` with attributes `username` and `role` and a method `permissions()` that returns an empty list. Create classes `Admin`, `Editor` and `Viewer` inheriting from `User`, each of which returns a different list of permissions. Write a function `check_access(user, required_permission)` that checks whether a given user has the required permission, using polymorphism.

13. Create an abstract class `Validator` (using the `abc` module) with an abstract method `validate(data)`. Create classes `EmailValidator`, `PhoneValidator` and `AgeValidator` that implement `validate()` and return `True` or `False` depending on the correctness of the data. Write a function `run_validations(validators, data)` that runs all validators and collects the results.

14. Define a class `Plugin` with methods `activate()` and `process(data)`. Create at least three derived classes representing different plugins (e.g. `UpperCasePlugin`, `TrimPlugin`, `ReplacePlugin`). Write a class `Pipeline` that stores a list of plugins and has a method `run(data)` that passes the data through the successive plugins polymorphically.

15. Create an example of multiple inheritance: a class `Flyable` with a method `fly()`, a class `Swimmable` with a method `swim()`, and a class `Duck` that inherits from both. Examine the method resolution order (MRO) using `Duck.__mro__` and write a short comment explaining the result.

16. Define a class `EventEmitter` with methods `on(event_name, callback)` and `emit(event_name, *args)`. Create a class `ChatRoom` inheriting from `EventEmitter` that adds methods `join(user)`, `leave(user)` and `send_message(user, text)`, each of which emits the corresponding event. Then create a class `ModeratedChatRoom` inheriting from `ChatRoom` that overrides `send_message()` and filters out forbidden words before sending.

17. Create a serialization system: a class `Serializer` with methods `serialize(obj)` and `deserialize(data)`. Create classes `JsonSerializer`, `CsvSerializer` and `XmlSerializer` that implement these methods. Then write a class `DataStore` that takes an object of type `Serializer` in its constructor (composition) and uses it to save and load data — compare this approach with inheritance and describe in a comment when composition is better.

18. Define a metaclass `SingletonMeta` that makes every class using it able to have only one instance. Create a class `DatabaseConnection` using this metaclass. Then create a class `PooledDatabaseConnection` inheriting from `DatabaseConnection` and check whether the singleton mechanism also works for the derived class. Describe the result in comments.

20 Text Representations of Objects

1. Define a class `Dog` with attributes `name` and `breed`. Implement the `__str__` method so that it returns text in the format `name (breed)`.
2. Define a class `Point` with attributes `x` and `y`. Implement the `__repr__` method so that it returns text that allows the object to be recreated, e.g. `Point(3, 5)`.
3. Define a class `Book` with attributes `title` and `author`. Implement both methods — `__str__` and `__repr__` — so that they return different representations of the same object.
4. Create a class `Temperature` with an attribute `celsius`. The `__str__` method should return the value with its unit, e.g. `36.6°C`, and `__repr__` — the constructor expression `Temperature(36.6)`.
5. Define a class `Color` with attributes `r`, `g`, `b`. Implement `__repr__` so that it satisfies the condition `eval(repr(obj)) == obj` (this also requires defining `__eq__`).
6. Create a class `Student` with attributes `first_name`, `last_name` and `grade`. Implement `__str__` returning a friendly description of the student, and `__repr__` — an unambiguous technical representation.
7. Write a class `Fraction` with attributes `numerator` and `denominator`. The `__str__` method should display the fraction in the format `3/4`, and `__repr__` — in the format `Fraction(3, 4)`.
8. Define a class `Product` with attributes `name` and `price`. Put several objects of this class into a list, and then print the list using `print()`. Observe which method — `__str__` or `__repr__` — is called for the elements of the list.
9. Create a class `Vector2D` with attributes `x` and `y`. Implement `__repr__` and `__str__`, and then check what `f"{obj}"`, `str(obj)` and `repr(obj)` return, as well as what is shown when the object is displayed in the interactive interpreter.
10. Write a class `LogEntry` with attributes `timestamp`, `level` and `message`. The `__str__` method should return a readable log entry, and `__repr__` — a full representation with all attributes, useful for debugging.
11. Define a class `Currency` with attributes `amount` and `code` (e.g. `"PLN"`, `"USD"`). Implement `__str__` formatting the amount with the currency symbol, and `__repr__` — with full decimal precision.

12. Create a class `Matrix` that stores a two-dimensional list of numbers. Implement `__str__` so that it displays the matrix in a readable row-by-row form, and `__repr__` — as a single-line constructor expression.

13. Define a base class `Shape` with a `__repr__` method that automatically reads the class name from `type(self).__name__`. Then create derived classes `Circle` and `Rectangle` that inherit this implementation and add their own attributes.

14. Write a class `LinkedList` with nodes of type `Node`. Implement `__repr__` for the `LinkedList` class so that it displays the entire contents of the list in the format `LinkedList(1 -> 2 -> 3)`.

15. Create a class `Person` with an attribute `secret_code` that should not appear in the output of `__str__` (shown to the user) but should be visible in `__repr__` (intended for the programmer).

16. Define a class `Config` that stores a dictionary of settings. Implement `__repr__`, which displays the sorted keys and their values, and `__str__`, which displays only the number of keys and the names of the first three.

17. Write a class decorator `auto_repr` that automatically generates a `__repr__` method based on the attributes passed to `__init__`. Test it on at least two different classes.

18. Create a class hierarchy `Animal` -> `Mammal` -> `Cat`. At each level, implement `__repr__` that calls `super().__repr__()` and appends the attributes specific to that level. Analyze the resulting representation string.

19. Define a class `LazyRepr` in which the `__repr__` method performs an expensive operation (e.g. simulated with `time.sleep`). Then write an optimized version that caches the result of `__repr__` using the memoization technique and compare the execution times.

20. Write a metaclass `ReprEnforcerMeta` that, when a class is created, checks whether the class defines both `__str__` and `__repr__`. If either of these methods is missing, the metaclass should raise a `TypeError` exception with an appropriate message.

21 Container Emulation

1. Define a class `MyList` that wraps a regular list. Implement the `__len__` method, which returns the number of elements, and test how the built-in `len()` function works on an object of this class.
2. Create a class `Sentence` that stores text as a string. Implement `__len__` so that it returns the number of words in the sentence rather than the number of characters.
3. Define a class `ColorPalette` that stores a list of colors. Implement `__getitem__` so that colors can be retrieved by index, e.g. `palette[2]`.
4. Extend the `ColorPalette` class with a `__setitem__` method that allows replacing the color at a given index, e.g. `palette[0] = "red"`.
5. Write a class `Matrix` that stores data in a flat list. Implement `__getitem__` taking a tuple `(row, col)`, so that element access looks like `matrix[2, 3]`.
6. Create a class `DefaultDict` that emulates a dictionary with a default value. Implement `__getitem__` so that for a non-existent key it returns the value given at initialization instead of raising a `KeyError`.
7. Define a class `Scoreboard` that stores player scores in a dictionary. Implement `__getitem__`, `__setitem__` and `__len__`, and then test the operations `board["Alice"]`, `board["Bob"] = 10` and `len(board)` on it.
8. Write a class `ImmutableList` that implements `__getitem__` and `__len__` but deliberately does not define `__setitem__`. Check which exception is raised when you try to assign a value to an element.
9. Create a class `RangeMap` in which `__getitem__` takes an integer and returns the name of the range that the number belongs to (e.g. `0-10` is `"low"`, `11-20` is `"medium"`). The ranges are defined at initialization.
10. Define a class `CircularBuffer` with a fixed size. Implement `__getitem__`, `__setitem__` and `__len__` so that indices beyond the size of the buffer automatically wrap around cyclically.
11. Write a class `SparseVector` that stores only the non-zero elements in a dictionary. Implement `__getitem__` (returning `0` for missing indices), `__setitem__` (removing the key when the value is set to `0`) and `__len__` (returning the logical size of the vector).

12. Create a class `History` that emulates a container with the ability to undo changes. The `__setitem__` method should push the previous value of the key onto a stack, and an additional method `undo(key)` should restore the last value.

13. Define a class `SliceableString` that implements `__getitem__` with full support for slices. The expression `obj[1:5]` should return a new `SliceableString` object, and `obj[0]` — a single character.

14. Write a class `Table` with data in the form of a list of dictionaries. Implement `__getitem__` so that it supports three access modes — by row index (int), by column name (string) and by a tuple `(row_index, column_name)`.

15. Create a class `ValidatedList` in which `__setitem__` checks the type of the inserted value and raises a `TypeError` if it does not match the type given at initialization (e.g. `ValidatedList(int)`).

16. Define a class `NestedDict` that allows access to nested data using a key in the form of a dotted path, e.g. `config["database.host"]` instead of `config["database"]["host"]`. Implement both `__getitem__` and `__setitem__`.

17. Write a class `TimeSeries` that stores `(datetime, value)` pairs. Implement `__getitem__` so that it accepts both a numeric index and a `datetime` object (looking up the nearest point in time), as well as a slice with `datetime` objects as the bounds of the range.

18. Create a class `ChainMap` that takes several dictionaries in its constructor. Implement `__getitem__` searching the dictionaries in order, `__setitem__` always writing to the first dictionary, and `__len__` returning the number of unique keys across all dictionaries.

19. Define a class `DatabaseTable` that emulates a database table. Implement `__getitem__` supporting queries in the style of `table[lambda row: row["age"] > 18]`, returning a new `DatabaseTable` object with the filtered rows. Add `__len__` and `__setitem__` for updating the rows that satisfy a condition.

20. Write a class `TensorView` that wraps a one-dimensional list of data and interprets it as a tensor of a given shape (e.g. `(2, 3, 4)`). Implement `__getitem__` with support for multidimensional indexing and slicing along each dimension, `__setitem__` with analogous logic, and `__len__` returning the size of the first dimension. Make sure that the view shares data with the original.

22 Arithmetic Emulation

1. Write a class `Temperature` with an attribute `celsius`. Implement the `__add__` method, which makes it possible to add two `Temperature` objects — the result should be a new object with the sum of the values.
2. Write a class `Counter` that stores an integer. Implement the `__add__` and `__sub__` methods so that `Counter` objects can be added to and subtracted from each other using the `+` and `-` operators.
3. Write a class `Vector2D` with attributes `x` and `y`. Implement the `__mul__` method, which allows multiplying a vector by a scalar (a number), returning a new vector with scaled coordinates.
4. Write a class `Price` that stores an amount as a `float`. Implement the `__truediv__` method so that the `/` operator divides the price by a given number and returns a new `Price` object.
5. Write a class `Weight` with an attribute `kg`. Implement the `__add__` and `__sub__` methods with support for adding/subtracting both another `Weight` object and a plain number (a scalar).
6. Write a class `Wallet` with an attribute `balance`. Implement `__floordiv__` (the `//` operator), which performs integer division of the balance by a given number, and `__mod__` (the `%` operator), which returns the remainder of the division.
7. Write a class `DataSeries` that stores a list of numeric values. Implement `__add__`, which performs element-by-element (elementwise) addition of two `DataSeries` objects of the same length. If the lengths do not match, raise a `ValueError`.
8. Write a class `Score` with an attribute `points`. Implement `__radd__` so that the expression `10 + score_obj` correctly delegates the operation to the `Score` object — test that both `score + 10` and `10 + score` work.
9. Write a class `Matrix1D` that stores a one-dimensional list of values. Implement `__mul__` handling two scenarios: multiplication by a scalar and the element-by-element product with another `Matrix1D` object. Add `__rmul__` for expressions such as `3 * matrix_obj`.
10. Write a class `Measurement` with attributes `value` and `unit`. Implement `__add__` and `__sub__`, which allow combining only measurements with the same `unit` — if the units do not match, raise a `TypeError` with a message describing the conflict.

11. Write a class `Percentage` with an attribute `value` (e.g. 75 means 75%). Implement `__mul__` so that `Percentage(50) * 200` returns a `float` equal to 100.0 (calculating a percentage of a number), and multiplying two `Percentage` objects returns a new `Percentage` object with the compound percentage.

12. Write a class `BankAccount` with an attribute `balance`. Implement `__iadd__` and `__isub__` (the `+=` and `-=` operators) so that they modify the balance in place. In the case of `__isub__`, prevent the balance from going below zero by raising a `ValueError`.

13. Write a class `ComplexNumber` with attributes `real` and `imag`. Implement the `__add__`, `__sub__` and `__mul__` methods according to the rules of complex number arithmetic. Also implement `__repr__` so that the result is displayed in the format `a + bi`.

14. Write a class `Polynomial` that represents a polynomial as a list of coefficients (the list index corresponds to the power). Implement `__add__` (adding polynomials) and `__mul__` (multiplying polynomials by a scalar). Pay attention to polynomials of different degrees.

15. Write a class `TimeDelta` with an attribute `seconds`. Implement `__add__`, `__sub__`, `__mul__` (multiplication by a scalar) and `__truediv__`. The `/` operator should distinguish division by a number (returns a `TimeDelta`) from division by another `TimeDelta` (returns a `float` — the ratio of the durations).

16. Write a class `Vector3D` with attributes `x`, `y`, `z`. Implement `__add__`, `__sub__`, `__mul__` (the dot product with another vector returning a `float`, or scaling the vector by a number), `__rmul__` and `__neg__` (the unary `-` operator). Use type hinting and `isinstance` to distinguish between operand types.

17. Write a class `Currency` with attributes `amount` and `code` (e.g. "PLN", "USD"). Implement `__add__`, `__sub__`, `__mul__` (by a scalar) and `__truediv__` (by a scalar). Addition and subtraction between different currencies should be forbidden (raise a `TypeError`). Add the `__radd__` and `__rmul__` methods for commutativity with scalars.

18. Write a class `FractionNumber` with attributes `numerator` and `denominator`. Implement `__add__`, `__sub__`, `__mul__` and `__truediv__` with full fraction arithmetic (including bringing fractions to a common denominator). Each operation should return the fraction in reduced form — use `math.gcd` to simplify the result.

19. Write a class `SparseVector` that stores a sparse vector as a dictionary (the keys are the indices of the non-zero elements). Implement `__add__`, `__sub__` and `__mul__` (the dot product with another `SparseVector` returning a `float`, or scaling by a number). Make sure the keys are merged correctly and zeros are removed from the resulting dictionary.

20. Write a family of classes representing physical units: `Distance` (meters), `Time` (seconds) and `Speed` (meters per second). Implement the arithmetic operators so that `Distance / Time` returns `Speed` and `Speed * Time` returns `Distance`. Also handle adding and subtracting objects of the same type and multiplication by a scalar. For invalid operations (e.g. `Distance + Time`), raise a `TypeError`.

23 Comparison Emulation

1. Write a class Temperature with a degrees attribute. Implement the `__eq__` method, which compares two objects of this class based on the value of degrees. Test the `==` operator on two instances.

2. In a class Product with attributes name and price, implement the `__lt__` method so that the `<` operator compares products by price. Create three objects and check the result of the comparison.

3. Extend the Product class from the previous task with the `__le__` method. Make sure the method correctly handles the case where the prices are equal.

4. Write a class Student with attributes name and grade. Implement the `__eq__` and `__lt__` methods, then sort a list of students using the built-in `sorted()` function.

5. In a class DataPoint with attributes label and value, implement the `__eq__` method so that it returns `NotImplemented` when other is not an instance of DataPoint. Explain in a comment why we return `NotImplemented` instead of `False`.

6. Write a class Rectangle with attributes width and height. Implement the `__eq__`, `__lt__` and `__gt__` methods so that comparisons are based on the area of the rectangle.

7. Use the `@functools.total_ordering` decorator in a class Score with a points attribute. Define only `__eq__` and `__lt__`, then test all six comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`).

8. Write a class SemanticVersion with attributes major, minor and patch. Implement the `__eq__` and `__lt__` methods so that they compare versions lexicographically (first major, then minor, then patch). Sort a list of versions.

9. In a class Interval with attributes start and end, implement the `__eq__` method comparing both endpoints of the interval and `__lt__` comparing intervals by their length. Check that two intervals with different endpoints but the same length are not equal.

10. Write a class Employee with attributes name, department and salary. Implement `__eq__` comparing by name and department (without salary) and `__lt__` comparing by salary only. In a comment, consider the consequences of such an inconsistent definition of equality and ordering.

11. Write a class Vector2D with attributes x and y. Implement `__eq__` comparing both components. Then define `__lt__` comparing vectors by their Euclidean length. Add type hints to all methods.

12. In a class Exam with attributes subject and score, implement `__eq__` and `__lt__`. Then create a list of Exam objects and find its maximum element using the built-in `max()` function — without passing the key parameter.

13. Write a class TemperatureReading with a celsius attribute. Implement the `__eq__` and `__lt__` methods, which compare temperatures with a given tolerance epsilon (e.g. 0.01). Two temperatures that differ by less than epsilon should be considered equal.

14. Create a class Money with attributes amount and currency. Implement `__eq__` and `__lt__` so that they raise a ValueError exception when we try to compare amounts in different currencies. Test both scenarios — matching and mismatching currencies.

15. Write a class Card with attributes rank and suit, representing a playing card. Define a dictionary with the order of ranks (e.g. "2" to "A") and implement `__eq__` and `__lt__` taking this order into account. Use `@total_ordering`. Sort a deck of five cards.

16. In a class TimeSeries storing a list of values, implement `__eq__`, which compares series element by element, and `__lt__`, which compares series by their arithmetic mean. Handle the case of series of different lengths in `__eq__` (they should be unequal).

17. Write a class Matrix2x2 storing a 2x2 matrix as a list of lists. Implement `__eq__` comparing element by element. Define `__lt__` comparing matrices by their determinants. Add a `__repr__` method that displays the matrix in a readable way.

18. Create a class hierarchy: a base class Shape and derived classes Circle and Square. In the Shape class, define the abstract method area(). Implement `__eq__` and `__lt__` in the Shape class, comparing by area(). Check that comparing a Circle with a Square works correctly thanks to inheritance.

19. Write a class CacheEntry with attributes key, value and priority. Implement `__eq__` comparing by key and `__lt__` comparing by priority. Then use the heapq module to build a priority queue based on CacheEntry objects — heapq requires a correctly defined `<` operator.

20. Write a (simplified) class DataFrame storing a dictionary of columns (name -> list of values). Implement `__eq__` checking the equality of structure and data column by column. Define `__lt__` comparing frames by the number of rows. Additionally, implement `__hash__` so that DataFrame objects are unhashable (as in pandas) — assign `__hash__ = None` — and explain in a comment why changing `__eq__` requires careful consideration of `__hash__`.

Part III

Advanced Topics

24 Decorators

In addition to defining each decorator, use it at least once.

1. Write a decorator `call_counter` that counts how many times a given function has been called. Store the counter as an attribute of the wrapper function. After each call, print the current value of the counter to the console.
2. Write a decorator `log_call` that prints the function's name before calling it and prints the returned value after the call. Use the `__name__` attribute of the original function.
3. Write a decorator `timer` that measures the function's execution time in seconds (the `time` module) and prints it when the function finishes. Return the result of the original function unchanged.
4. Write a decorator `suppress_errors` that catches all exceptions raised by the function, prints a message with the error text and returns `None` instead of propagating the exception.
5. Write a decorator `repeat(n)` that takes an argument `n` and causes the decorated function to be called `n` times with the same arguments. The decorator returns the result of the last call.
6. Write a decorator `cache_result` that stores the results of function calls in a dictionary, using the positional arguments as the key. If the function has already been called with the same arguments, return the result from the dictionary without calling it again.
7. Write a decorator `validate_types` that takes types as positional arguments (e.g. `@validate_types(int, int)`). Before calling the function, check whether the passed arguments have the corresponding types. If not, raise `TypeError` with a descriptive message.
8. Write a decorator `retry(max_attempts, delay)` that, when an exception occurs, retries the function call up to `max_attempts` times, waiting `delay` seconds between attempts. If all attempts fail, raise the last exception.
9. Write a decorator `flatten_result` that, after calling the function, checks whether the result is a nested list (a list of lists). If so, it flattens it into a single-level list. If the result is not a list, it returns it unchanged.
10. Write a decorator `sort_result(key=None, reverse=False)` that sorts the function's result if it is a list. The `key` and `reverse` parameters work the same way as in the built-in `sorted` function. If the result is not a list, return it unchanged.

11. Write a decorator `transform_keys(func)` that takes a transforming function (e.g. `str.upper`). If the decorated function returns a dictionary, apply the transformation to all keys of that dictionary. If the result is not a dictionary, return it unchanged.

12. Write a decorator `ensure_keys(*required_keys)` that checks whether the dictionary returned by the decorated function contains all the required keys. If any key is missing, raise `KeyError` with a list of the missing keys. If the result is not a dictionary, raise `TypeError`.

13. Write a decorator `paginate(page_size)` that takes a page size. If the decorated function returns a list, the decorator turns it into a list of sublists, each of which has at most `page_size` elements. If the result is not a list, return it unchanged.

14. Write a decorator `filter_result(predicate)` that takes a filtering function. If the decorated function returns a list, the decorator keeps only those elements for which `predicate` returns `True`. If the result is not a list, return it unchanged.

15. Write a decorator `default_args(**defaults)` that takes default values for keyword arguments. If the decorated function is called without any of these arguments, the decorator adds it automatically. Use the `inspect` module to examine the function's signature.

16. Write a decorator `audit_log(log_list)` that takes an external list as an argument. After each call of the decorated function, it appends to this list a dictionary containing the keys: `function_name`, `args`, `kwargs`, `result` and `timestamp` (use `datetime.datetime.now()`).

17. Write a decorator `compose(*decorators)` that takes any number of other decorators and applies them in turn to the decorated function (the first decorator in the list wraps the function first, the last one last). Test it by combining at least two of the decorators written earlier.

18. Write a decorator `rate_limit(max_calls, period)` that allows at most `max_calls` calls of the decorated function within `period` seconds. If the limit is exceeded, raise a `RuntimeError` exception with information on how many seconds remain until it is unblocked.

19. Write a decorator `dispatch_on_type` that registers different implementations of a function depending on the type of the first argument. The decorator exposes a `register(type, func)` method on the wrapper function. When called, it checks the type of the first argument and delegates to the appropriate registered implementation. If the type is not registered, it calls the default function.

20. Write a decorator `pipeline(*transformers)` that takes any number of transforming functions. After calling the decorated function, it passes its result through each transformer in turn. If any transformer raises an exception, stop the chain, print a diagnostic message indicating which transformer failed, and return the result from before that step.

25 Regular Expressions

1. Import the `re` module and use `re.findall()` to find all floating-point numbers in the text `"Price: 49.99 PLN, discount: 10.50 PLN"`, using the pattern `r"\d+\.\d+"`.
2. Explain the difference between writing the pattern as `"\\d+\\.\\d+"` and as `r"\d+\.\d+"`. Which one is more readable and why?
3. Given the string `text = "Report_2024 report_2025 REPORT_2026"`, use `re.findall()` with a pattern containing `\d{4}` to extract all four-digit numbers from it.
4. Given the string `log = "ERROR: file not found at /data/input.csv"`, write a pattern with `r"\b...\b"` that searches it for exactly the word `file` as a whole word (not as part of another word).
5. Given the list `files = ["data.csv", "report.pdf", "sales.xlsx", "model.pkl"]`, write an expression with `re.search()` and a pattern using the alternation `(csv|xlsx)` and the anchor `$` that filters out only the files with the csv or xlsx extension.
6. Given the string `record = "  John Smith  (b. 1990) "` (`b.` is the abbreviation for "born"), use `re.sub()` with an appropriate pattern in an r-string to remove the parenthesized fragment containing the year of birth.
7. Given the string `messy = "Anna   BROWN   (b.  1985)"`, apply `re.sub(r"\s+", " ", messy)` to replace multiple whitespace characters with a single space. Describe what the `+` quantifier does here.
8. Write a regular expression in an r-string that matches an optional dot after the abbreviation `b` (for "born") — use the `?` quantifier in the pattern `r"\bb\.?"`.
9. Define the variable `column = "price"` and use an rf-string to create the pattern `rf"\b{column}\b"`. Then use `re.findall()` on the text `"The total_price and price are different"` and explain why the result contains only `"price"`.
10. Given the list of extensions `extensions = ["csv", "xlsx", "parquet"]`, dynamically build a regex pattern using an rf-string and the `join()` method so that the resulting pattern is `r"\.(csv|xlsx|parquet)$"`.
11. Given the string `data = "Height: 180cm, weight: 75kg, age: 30yrs"`, write a pattern with `re.findall()` that extracts all numbers (sequences of digits), using `r"\d+"`.

12. Given the list of records `raw_data = ["John Smith (b. 1990)", "Anna Brown (b. 1985)"]`, write a `for` loop in which you use `re.sub()` to remove the fragment with the year of birth and then remove redundant spaces — as in the lecture example.

13. Write a pattern in an r-string that matches any sequence of whitespace characters (`\s+`) surrounded by word boundaries (`\b`). Test it on the string `"Hello  World"` using `re.sub()`, replacing multiple spaces with a hyphen.

14. Given the string `line = "error: timeout at 12:45:30"`, use `re.search()` with the pattern `r"\d{2}:\d{2}:\d{2}"` to find the timestamp in it. Explain the role of the `{2}` quantifier in the pattern.

15. Given the list of files `all_files = ["notes.txt", "data.csv", "image.png", "report.xlsx", "archive.parquet"]`, write a list comprehension with `re.search()` and a dynamically built rf-pattern (based on a list of extensions) that returns only the data files.

16. Given the text `info = "PETER  Wilson(b.2000)"`, apply two `re.sub()` operations to it in turn: the first to remove the fragment with the year of birth (note the absence of a space before the parenthesis), the second to normalize the spaces. Finally, call `.strip().title()`.

17. Write a pattern in an r-string that matches e-mail addresses in a simplified form: a sequence of alphanumeric characters, the `@` sign, a sequence of alphanumeric characters, a dot and a domain. Use `\w+`, a literal dot `\.` and `re.findall()`.

18. Define a list of banned words `banned = ["error", "fail", "crash"]`. Using an rf-string and the `join()` method, create a pattern with alternation and word boundaries `\b`, then use `re.findall()` to search for these words in a sample log.

19. Given a multiline string with data in the format `"key: value"`, write a pattern with `re.findall()` using a group `()` and `\s+` that extracts only the values (after the colon and space). Use an r-string.

20. Write a function `clean_records(records)` that takes a list of strings in the format `"FIRSTNAME LASTNAME (b. YYYY)"`. Using `re.sub()` with an r-string, the function should remove the year of birth in parentheses, normalize the spaces (`\s+`), trim the string (`.strip()`) and apply `.title()`. Return the list of cleaned records. Use an rf-string so that the pattern handles an optional space and an optional dot after `b`.

26 Iterators and Generators

1. Write a class `CountdownIterator` that takes an integer `start` and, when iterated over, returns successive numbers from `start` down to 0 inclusive. Use the iterator protocol (`__iter__`, `__next__`).
2. Create a generator expression `cubes` that produces the cubes of successive numbers for `range(20)`. Then use `sum()` to calculate their sum.
3. Implement a class `RangeIterator` with parameters `start`, `stop` and `step` that works analogously to the built-in `range`, but fully uses the iterator protocol.
4. Write a generator function `even_numbers(limit)` that returns successive even numbers less than `limit`. Use it in a `for` loop to print the values.
5. Using a list comprehension, transform the list `raw_data = ["12", "abc", "7.5", "", "3", "xyz"]` into a list of integers containing only the valid integer values.
6. Write a class `FibonacciIterator` that returns the next number of the Fibonacci sequence on each call of `__next__()`. The iterator should stop after generating `n` elements (`StopIteration`).
7. Create a generator function `chunked(data, chunk_size)` that works similarly to the `DataBatchIterator` class from the lecture, but uses the `yield` statement instead of a class.
8. Write a dictionary comprehension `word_lengths` that, for the list `words = ["python", "data", "iterator", "yield", "set"]`, returns a mapping word → its length, but only for words longer than 3 characters.
9. Implement a generator `running_average(numbers)` that, for a list of numbers, returns successive values of the running average (after the first, second, third… element).
10. Write a class `ReverseIterator` that takes a sequence and, when iterated over, returns its elements in reverse order. Do not use the `reversed()` function.
11. Create a generator pipeline for the list of strings `lines = ["10", "abc", "20", "", "30"]`: the first generator filters the values that are numbers, the second converts them to `int`, the third squares them. Calculate the final sum using `sum()`.
12. Write a generator function `cycle_n_times(iterable, n)` that iterates over the given iterable `n` times, returning successive elements.

13. Using a generator expression and the `any()` function, check whether the list `numbers = [3, 7, 11, 14, 19, 25]` contains any number divisible by 5. Then use `all()` to check whether all of them are less than 30.

14. Implement a class `BatchIterator` with type hinting (use `Iterator`, `List`, `Any` from the `typing` module) that returns data in batches of a given size, but additionally skips batches containing fewer than `min_size` elements.

15. Create a set comprehension `unique_first_letters` containing the first letters (lowercase) of the words from the list `words = ["Apple", "banana", "apricot", "Cherry", "blueberry"]`, but only for words at least 5 characters long.

16. Write a generator function `read_lines_filtered(filepath, keyword)` that reads a file line by line and returns only those lines that contain the given keyword (after `strip()`). Use the `with open(...)` statement.

17. Implement a class `ZipIterator` that takes two sequences and, when iterated over, returns tuples of pairs of corresponding elements. Iteration ends when the shorter of the sequences is exhausted. Do not use the built-in `zip()`.

18. Write a generator pipeline for processing a CSV file (analogous to the lecture example): `load_records` reads the rows, `filter_valid` rejects records with a missing value in column 1, `extract_feature` returns numeric values from a selected column. Then write a function `compute_stats(pipeline)` that returns a dictionary with the keys `min`, `max`, `mean`. Use type hinting (`Generator`, `List`, `Dict`).

19. Using a nested list comprehension, generate a 5×5 multiplication table as a list of lists. Then use a generator expression to calculate the sum of all elements on the diagonal.

20. Implement a class `MovingWindowIterator` with type hinting that takes a sequence `data: List[Any]` and `window_size: int`. When iterated over, it returns successive sublists (windows) of the given size, shifted by one element. Once all possible windows are exhausted, it raises `StopIteration`. Additionally, write a generator expression that uses this class to calculate the mean in each window for a list of numbers.

27 Bitwise Operators

1. Write a function `is_even` that takes an integer and returns a Boolean value. Using the `&` operator, the function should check whether the number is even. The function should also work correctly for negative numbers.
2. Write a function `last_bit` that takes an integer and returns the value of its least significant bit (0 or 1).
3. Write a function `multiply_by_power_of_two` that takes two integers `x` and `n` and returns the result of multiplying `x` by `2^n` using a bit shift. Then write an analogous function `divide_by_power_of_two` that performs integer division.
4. Write a function `mod_power_of_two` that takes two integers `x` and `n` and returns the remainder of dividing `x` by `2^n`, using only bitwise operators (without `%`).
5. Write a function `get_bit` that takes an integer `x` and a position `i` (an integer) and returns the value of the bit at position `i` in the number `x` (counting from position 0 as the least significant).
6. Write three functions: `set_bit`, `clear_bit` and `toggle_bit`. Each of them takes an integer `x` and a position `i` (an integer) and returns an integer — with the bit at position `i` set, cleared and toggled, respectively.
7. Write a function `to_u2` that takes an integer `x` and a number of bits `bits` (an integer) and returns the two's complement (U2) representation of `x` at the given width as a string of zeros and ones.
8. Write a function `count_ones` that takes a non-negative integer and returns an integer equal to the number of bits set to 1. Use the `bin` function and the `count` method.
9. Write a function `hamming_distance` that takes two integers and returns an integer equal to the Hamming distance between them (the number of positions at which their bits differ).
10. Write a function `is_power_of_two` that takes an integer and returns a Boolean value indicating whether this number is a positive power of two. Use the `x & (x - 1)` pattern.
11. Write a function `lowest_set_bit` that takes an integer `x` and returns an integer equal to the value of the lowest set bit (e.g. for the input `44` it returns `4`). Use an expression with `-x`.

12. Write a function `find_unique` that takes a list of integers and returns an integer — the only value that appears an odd number of times, when all the others occur in pairs. Use XOR and a `for` loop.

13. Write a function `xor_swap` that takes two integers `a` and `b` and returns a tuple of two integers with the values swapped. Do not use a temporary variable — use only the `^` operator.

14. Write a function `reverse_bits` that takes an integer `x` and a number of bits `bits` (an integer) and returns the integer obtained by reversing the order of bits in the `bits`-bit representation of `x` (e.g. for `x = 0b1011` and `bits = 4` the result is `0b1101`).

15. Write a function `extract_byte` that takes an integer `x` and a byte number `n` (an integer) and returns an integer equal to the value of the `n`-th byte of `x` (byte zero is the least significant 8 bits). Use a combination of `>>` and the mask `& 0xFF`.

16. Define a set of permission flags using `IntFlag` and `auto`: `READ`, `WRITE`, `EXECUTE`, `ADMIN`. Write three functions: `grant`, which takes the current permissions and a new flag and returns the new permissions; `revoke`, which takes the current permissions and a flag to remove and returns the new permissions; and `has_all`, which takes the current permissions and the required flags and returns a Boolean value indicating whether the current permissions contain all the required flags.

17. Write a class `Bitmask` with fields `value` (an integer) and `width` (an integer) and the methods: `set` (takes a bit position, returns nothing), `clear` (takes a bit position, returns nothing), `flip` (takes a bit position, returns nothing), `is_set` (takes a bit position, returns a Boolean value) and `__repr__`, which returns a binary representation of width `width` padded with leading zeros.

18. Extend the `BitSet` class from the lecture with a `difference` method, which takes another `BitSet` object and returns a new `BitSet` containing the elements present in `self` but absent from `other`. Also add an `is_subset` method, which takes another `BitSet` object and returns a Boolean value indicating whether `self` is a subset of `other`.

19. Write a class `BloomFilter` with the methods: `__init__`, which takes a size (an integer) and returns nothing; `add`, which takes a value (an integer) and returns nothing; and `__contains__`, which takes a value (an integer) and returns a Boolean value. The filter should store its data in a single field `bits` (an integer) and use three simple hash functions of the form `(value * k) & (size - 1)` for `k = 1, 3, 7`. The `add` method sets the corresponding bits, and the `in` operator checks whether all these bits are set. Assume that the size is a power of two.

20. Write a class `Xorshift32` implementing a pseudorandom number generator from the xorshift family. In the `__init__` method (takes a seed that is an integer, returns nothing), store the seed in the field `state` (an integer). The `next` method (takes no arguments, returns an integer) should perform, in order: `state ^= (state << 13) & 0xFFFFFFFF`, `state ^= (state >> 17)`, `state ^= (state << 5) & 0xFFFFFFFF`, and then return

the new value of the `state` field masked to 32 bits. Add a `random_below` method, which takes an integer `n` and returns an integer in the range `[0, n)`, assuming that `n` is a power of two.

28 Selected Sequence Types

1. Write a function `to_bytes_literal` that takes an ASCII string and returns the corresponding `bytes` object written as a literal with the `b` prefix. Then print the type of the result and its length.

2. Write a function `count_byte` that takes a `bytes` object and a single byte (an integer in the range 0–255) and returns an integer equal to the number of occurrences of that byte in the sequence.

3. Write a function `pack_three_doubles` that takes three floating-point numbers and returns a `bytes` object produced by serializing them with `struct.pack` using the format `"3d"`. Check the length of the result and make sure it is 24.

4. Write a function `unpack_three_doubles` that takes a `bytes` object of length 24 and returns a tuple of three floating-point numbers recovered using `struct.unpack`.

5. Write a function `pack_vector` that takes a list of floating-point numbers of any length and returns a `bytes` object with their serialization in the format `f"{len(values)}d"`. Then write a function `unpack_vector` that takes a `bytes` object and the number of elements (an integer) and returns a tuple of floating-point numbers.

6. Write a function `to_mutable` that takes a `bytes` object and returns the corresponding `bytearray` object. Also write a function `to_immutable` that performs the conversion in the other direction.

7. Write a function `replace_first_byte` that takes a `bytearray` object and a new byte value (an integer in the range 0–255), changes the first byte to the given value and returns nothing. The function should modify the buffer in place.

8. Write a function `overwrite_double` that takes a `bytearray` object containing a serialized vector of floating-point numbers, an element index (an integer) and a new value (a floating-point number), and overwrites the indicated element using `struct.pack_into` with the format `"d"`. The function returns nothing.

9. Write a function `build_int_array` that takes a list of integers and returns an `array.array` object with type code `'i'` (int32) containing these values.

10. Write a function `array_memory_usage` that takes an `array.array` object and returns an integer equal to the estimated size of the memory it occupies in bytes. Use `buffer_info` and the `itemsize` field.

11. Write a function `compare_memory` that takes an integer `n`, creates a regular list `list(range(n))` and an `array.array` with type code `'i'` containing the same elements, and then prints the memory size of both structures. For the list use `sys.getsizeof`; for the array use the product of the length and `itemsize`.

12. Write a function `save_floats` that takes an `array.array` object with type code `'d'` and a file path (a string), and then writes the contents of the array to a binary file using `tofile`. The function returns nothing.

13. Write a function `load_floats` that takes a file path (a string) and the number of elements to read (an integer) and returns an `array.array` object with type code `'d'` loaded from a binary file using `fromfile`.

14. Write a function `roundtrip_array` that takes a list of floating-point numbers and a file path (a string). The function should create an `array.array` with type code `'d'`, write it to the file, read it back from the file and return the array that was read.

15. Write a function `array_from_generator` that takes an integer `n` and returns an `array.array` object with type code `'d'` containing `n` values of the form `x * 0.1` for `x` ranging from 0 to `n-1`. Use a generator expression passed to the `array.array` constructor.

16. Write a function `read_pgm_header` that takes the path to a PGM file (a string) and returns a tuple containing the width, height and maximum pixel value (three integers). The function should open the file in `"rb"` mode, read the line with the magic number `b"P5"`, skip comment lines beginning with `b"#"` and not read the pixels themselves.

17. Write a function `read_pgm_pixels` that takes the path to a PGM file (a string) and returns a tuple: width (an integer), height (an integer), maximum value (an integer) and a `bytearray` object with the raw pixels. The function should verify the magic number `b"P5"` using an `assert` statement.

18. Write a function `pixels_to_matrix` that takes a `bytearray` object with the pixels of an image, the width (an integer) and the height (an integer), and returns a list of lists of integers representing the image row by row. Use sequence slicing and the `list` constructor.

19. Write a function `invert_pgm` that takes a `bytearray` object with the pixels of a PGM image and the maximum pixel value (an integer), modifies the buffer in place so that each pixel is replaced with the value `max_val - pixel` (the negative of the image), and returns nothing. Use a `for` loop over the indices.

20. Write a function `pgm_pipeline` that takes the path to an input PGM file (a string) and the path to an output file (a string). The function should read the header and pixels of the image (analogously to `read_pgm_pixels`), store the pixels in an `array.array` with type code `'b'` or `'B'`, write this array to a binary file using `tofile`, and then read it back using `fromfile` and return the array that was read.

29 Lambda Expressions

1. Write a lambda expression named `cube` that takes an integer and returns its cube. Call it for the value 4 and print the result.
2. Define a lambda expression `area_rectangle` that takes two arguments (length and width) and returns the area of the rectangle. Check how it works for several pairs of values.
3. Using the `map` function and a lambda expression, transform the list of integers `numbers` into a list of their absolute values. Print the result as a list.
4. Using `filter` and a lambda expression, select from the list `words` only those strings whose length is greater than 5. The result should be a new list.
5. Sort the list of tuples `points` of the form `(x, y)` in ascending order by the second coordinate, using the `sorted` function and the `key` parameter given as a lambda expression.
6. Given a list of dictionaries `students`, in which each element contains the keys `name` and `grade`, sort it in descending order by the value of `grade` using `sorted` and a lambda.
7. Using `functools.reduce` and a lambda expression, calculate the sum of squares of the elements of the list `values` containing floating-point numbers.
8. Using `reduce` and a lambda, find the largest element in the list of integers `numbers` without using the built-in `max` function.
9. Define a lambda expression `classify_temperature` that, for a given temperature (a floating-point number), returns one of the strings `"cold"`, `"warm"` or `"hot"`, depending on the range (below 10, between 10 and 25, above 25). Use a conditional expression.
10. For a list of strings `names`, use a combination of `map` and a lambda to convert each name to a format in which the first letter is uppercase and the rest are lowercase. Then, from the transformed list, filter out the names beginning with the letter `A`.
11. Given two lists of equal length — `prices` and `quantities` — use `zip`, `map` and a lambda to calculate a list of the products of corresponding elements.
12. Using `enumerate` and a lambda expression passed to `map`, create a list of strings of the form `"i: value"` for each position of the list `items`.
13. Write a generator expression `even_squares` that, for successive numbers in the range from 1 to 100, returns the squares of the even numbers. Then print the first five values using `next`.

14. Using the `iter` and `next` functions, manually retrieve the first three elements of the list `letters` containing letters of the alphabet. Explain in a comment the difference between an iterable and an iterator.

15. Define a function `make_multiplier` that takes a number `factor` and returns a lambda expression that multiplies its argument by this number. Demonstrate how it works by creating the functions `double` and `triple`.

16. Given a list of words `text_tokens`, use `sorted` together with a lambda to order them first in ascending order by word length and then alphabetically (use a tuple as the value returned by the key).

17. Using only `map`, `filter` and lambdas, write an expression that calculates the sum of cubes of the odd numbers in the range from 1 to 50. Use the `sum` function for the final summation.

18. Implement a generator function `running_average` that, for a given iterable of numbers, returns successive running averages. Then, using `map` and a lambda, round the results to two decimal places.

19. Given a list of dictionaries `transactions` with the keys `category` and `amount`, use `reduce` and a lambda to calculate a dictionary aggregating the sums of amounts within each category. The accumulator should be an empty dictionary.

20. Write a function `compose` that takes any number of single-argument functions (passed as lambda expressions) and returns their composition as a single lambda expression. Demonstrate how it works by composing three operations: adding 1, squaring and multiplying by 2.

30 The `functools` Module

1. Using `functools.reduce`, calculate the sum of the elements of the list of integers `values`. Set the initial value of the accumulator to 0.

2. Using `functools.reduce` and the `operator` module, calculate the product of the elements of the list `numbers` containing floating-point numbers. Do not use a lambda expression.

3. Write a function `clip_value` that takes a value, a lower limit and an upper limit and returns the value clipped to the given range. Then, using `functools.partial`, create a function `clip_percent` with the limits set to 0 and 100.

4. Define a function `format_number(value, precision, suffix)` that returns a formatted string. Using `functools.partial`, configure a variant `format_percent` with precision 2 and the suffix `"%"` and a variant `format_currency` with precision 2 and the suffix `" PLN"`.

5. Using the `functools.lru_cache` decorator, recursively implement a function `fibonacci(n)` that returns the n-th term of the Fibonacci sequence. Call the function for the value 100 and print the result.

6. Implement a recursive function `binomial(n, k)` that calculates the binomial coefficient. Use the `functools.cache` decorator, and then, after several calls, print the statistics from `cache_info`.

7. Using the `functools.lru_cache(maxsize=32)` decorator, implement a function `expensive_lookup(key)` that simulates an expensive data fetch by calling `time.sleep(1)` and returning a dictionary. Measure the time of the second call with the same argument and compare it with the first.

8. Explain in a comment why a function decorated with `lru_cache` will not work if we pass it an argument of type `list`. Then write a function `sum_unique(items)` that takes a tuple and returns the sum of its unique values, and demonstrate the use of the cache with different tuples as input.

9. Define a class `Statistics` with a constructor that takes a sequence of numbers. Use `functools.cached_property` to define the attributes `mean`, `variance` and `stdev`. Inside each method, add a `print` call reporting that the computation has started, in order to verify that it is performed only once.

10. Implement a class `TextAnalyzer` with a constructor that takes a string. Using `functools.cached_property`, add the attributes `word_count`, `unique_words` (returning a `set`) and `most_common` (returning the most frequently occurring word).

11. Using `functools.singledispatch`, define a function `describe(value)` that returns the string `"integer: ..."` for an argument of type `int`, the string `"float: ..."` for `float`, and the string `"string of length ..."` for `str`. Add a default implementation that raises `TypeError`.

12. Using `functools.singledispatch`, write a function `serialize(data)` that, for a list, returns its representation as a string with the elements separated by commas; for a dictionary, as `key=value` pairs separated by semicolons; and for a tuple, as the elements in round brackets.

13. Write a decorator `log_calls` that prints the name of the decorated function and the passed arguments before each call. Use `functools.wraps` to preserve the metadata of the original function. Check that `__name__` and `__doc__` of the decorated function are preserved.

14. Implement a decorator `retry(times)` that retries the function call up to `times` times if an exception is raised. Use `functools.wraps`. Test the decorator on a function that randomly either raises an exception or returns a result.

15. Using the `functools.total_ordering` decorator, implement a class `Temperature` that stores a value in degrees Celsius. Define only the `__eq__` and `__lt__` methods, and then check how all the comparison operators and the `sorted` function work on a list of instances.

16. Using `functools.total_ordering`, implement a class `Version` whose instances are compared based on the tuple `(major, minor, patch)`. Demonstrate the use of the `<`, `>=` and `==` operators and the sorting of a list of objects.

17. Using `functools.reduce` and the `operator.and_` function from the `operator` module, calculate the intersection of the list of sets `sets_list`. Then do the same for the union of the sets, using `operator.or_`.

18. Given a list of dictionaries `records`, in which each element contains the keys `category` and `amount`, use `functools.reduce` to build a dictionary aggregating the sums of amounts per category. The accumulator should be an empty dictionary.

19. Write a decorator `memoize_with_stats` that combines memoization with a hit and miss counter (without using `lru_cache`). Use `functools.wraps`. The decorator should add the methods `cache_hits` and `cache_misses` to the function, returning the current counters.

20. Using `functools.partial`, `functools.reduce` and `functools.singledispatch`, design a function `pipeline(value, *steps)` that applies a sequence of transformations to the input value. Then define a `singledispatch` function `apply_default` that provides different transformation steps for the types `int`, `str` and `list`, and use `partial` to configure reusable variants of the pipeline steps.